

Space Travel & Transport – Specification

v2.16

Format: Markdown with screenshots, SVG diagrams and mermaid diagrams **embedded directly in the file** (base64 images) — a single `.md` file, no companion folder required.

What's changed since v2.0 (DOCX): several developments described there as mere proposals are now **actually implemented and verified** in `space-travel.html` — the credits system, the F3/F10 shortcuts, the widened flight-plan panel, the roll fix, and above all the cinematic staging of the shuttles with a physical dock under the ship and a textured hull dressing. Like v2.0, this document clearly separates what exists (Part I) from what remains to be built (Part II) — but the boundary between the two has shifted.

Revision History

Version	Date	Author	Content
2.0	September 13, 2026	Frédéric Delorme	Initial specification (DOCX): overview, procedural universe, ship, controls, navigation, planetary systems, arrival sequence, HUD, architecture, i18n.
2.1	September 16, 2026	Frédéric Delorme	Converted to standalone Markdown (embedded images). Added proposals §14 (radio dialogues via embedded AI, Gemini Nano) and §15 (touch controls), then implemented and verified both. Fixes: dock flickering (z-fighting), off-center/shaky shuttle-launch camera, orbit insertion too fast, cargo trajectory able to pass through celestial bodies.
2.2	September 16–17, 2026	Frédéric Delorme	Tier 1 delivered: Pause mode (§11), HUD control bar (§12), port services (§16, new). First batch of fixes (systems spaced too tightly, planets missing from flight plan, radio/services overlap).
2.3	September 17, 2026	Frédéric Delorme	Second batch of fixes and improvements (§17): bugs (general panel overlap, touch look gone missing, icon-bar position, missing help button) and polish (close buttons, F1-F9+H shortcuts, dynamic camera icon, overlay help, radio welcome message, avatars, 3-phase shuttle launch trajectory, thruster trail, ship labels, object rendering in the "nearest object" panel). Proposal (not implemented) for the universe map (§19).
2.4	September 17, 2026	Frédéric Delorme	Touch look icon replaced with a camera icon. Two new panels (§21): visual itinerary (bottom right) — green/blue step track with a continuous position dot — and orientation relative to a Lagrange point (bottom left) — compass + distance. Two extra HUD icons, fixed shortcuts I / L . Screenshots not fully redone this time; a single screenshot added (§21) for the two new panels.

Version	Date	Author	Content
2.5	September 18, 2026	Frédéric Delorme	Three new proposals, not implemented — each deserving its own validation cycle before being built, like the universe map once did. Fuel (§22) : gauge, consumption tied to engine regime, refueling at stopovers, initial quantities sized against the route generator's real constants. Quantum propulsion and star map (§23) : purchase at a stopover of a jump module, destination selected on the universe map — the latter's design finally settled (option B from §19, now serving double duty). Orbital ports (§24) : a second port type, in orbit rather than on the ground, a small number of 3D mockups instanced across certain systems. Key conflict I / §13 (flagged in v2.4) resolved: engine fire moved to J .
2.6	September 18, 2026	Frédéric Delorme	Fuel (§22) implemented : gauge merged into the PROPULSION panel (option B chosen), consumption tied to engine regime, refueling at stopovers, empty-tank behavior settled (cruise cut to 20%, boost disabled). Bug found and fixed during implementation: possible overlap between the port services panel — now taller — and the itinerary panel on a short window.
2.7	September 19, 2026	Frédéric Delorme	Quantum propulsion and star map (§23) implemented : one-time purchase at a stopover (18,000 CR), 3D map on a dedicated scene (simple spheres rather than the full star meshes, lighter for ~24 candidates), raycast-based selection, three-phase jump sequence reusing the engines and the boost's <code>.speedLines</code> effect rather than a particle system. Two bugs found and fixed while implementing: an HTML comment mistakenly closed with JS syntax (<code>*/</code>), which made the map panel vanish from the DOM; the map overlay nested inside the general HUD container, hidden by mistake by its own screen-clearing rule.
2.8	September 19, 2026	Frédéric Delorme	Round of improvements and fixes to the star map (§23): star rendering brought back in line with the main game (same shader, halo, real color) at a reduced scale rather than plain spheres; a fine white reference grid with lettered marks on a horizontal plane centered on the displayed stars; dashed lines, colored per star, dropping each one to that plane; name labels; the ship shown as a miniature of the real model (visual-group clone, dedicated lighting added) with a ring+arrow label as in-game; very slow auto-rotation after 5s of mouse inactivity. Two bugs fixed: ESCAPE/SPACE/ENTER sent the player back to the title screen or resumed flight without properly closing the map (all three reuse <code>gamePaused</code> , shared with the general pause, without knowing it); the M key didn't respond on an AZERTY keyboard (<code>e.code</code> gives the key's physical position, where the letter M sits at the QWERTY semicolon's position).
2.9	September 19, 2026	Frédéric Delorme	Title screen (§25): keyboard navigation and background flythrough. Arrows + ENTER/SPACE to choose the language, alongside the click/touch already in place. A succession of very slow shots through the already-generated universe (a random star/planet/nebula), HUD and labels hidden. Three bugs found while implementing, all tied to assumptions that only held during normal flight: the star apparent-size calculation (never run outside the normal game loop); the framing distance chosen for a star (based on its halo, hundreds of units — the angle calculation is only meant for a flythrough a few units away); and an opaque title-screen background hiding an already-correct render underneath, the same flaw already found on the star map (§23.6).

Version	Date	Author	Content
2.10	September 19, 2026	Frédéric Delorme	Title screen (§25.2): the ship and its route trace are now hidden too during the background flythrough (<code>.visible = false</code> , restored once the game actually starts) — spotted from a screenshot clearly showing the route's dashed gate markers in the backdrop. HUD hiding, on the other hand, turned out to already be correct despite a first, misleading check (<code>getComputedStyle</code> on a panel doesn't reflect a hidden ancestor's effect — confirmed unambiguously via <code>offsetParent / getBoundingClientRect</code>).
2.11	September 19, 2026	Frédéric Delorme	Three combined requests. Backward dolly shot removed from the title screen (§25.2): each shot now approaches its target from the camera's current position, never from a random direction. Jerky ship movement fixed (§5): the nearest-point search on the route curve, too coarse (24 steps over 420u), froze <code>ROUTE.s</code> in steps before jumping all at once — replaced with a two-pass search (coarse then fine), more precise and cheaper. Background music and volume controls (§26): new two-slider dialog (music, synthesized voice), playback triggered on the first genuine user gesture to respect browser autoplay policy.
2.12	September 19, 2026	Frédéric Delorme	Title screen (§25.4): background stars re-centered on the flythrough camera (they scrolled too fast and passed in front of objects) and always rendered behind everything; "(c) 2026 - game by Frédéric Delorme - music by ScoreStudio from Envato" notice at the bottom of the screen; music starts as soon as the title screen is shown (§26.1), within the limits of browser autoplay policy.
2.13	September 24, 2026	Frédéric Delorme	Generated ships (§27): the single ship is replaced by the generator from the study "Generative construction of cargo ships" — ten models, from the 61 m light tug to the 237 m spine freighter — chosen on a temporary selection screen between the title screen and the game, then wired into every existing system (gauges, RCS, lights, beacons, shuttle dock and arm, camera). Shader-based Epstein drive on every model. Quantum jump v2 (§28): reserved for long-haul ships fitted with warp rings, new spacetime-distortion sequence (post-processing active only during the jump).
2.14	September 24, 2026	Frédéric Delorme	Selection screen (§27.2): propulsion-system selector on long-haul ships — "Epstein" or "Epstein + quantum jump" —, preview and sheet recomputed from the choice; option greyed out on other ships. A long-haul ship leaving without rings can buy the module in port: the rings are then fitted to the hull, without touching the dock or the arm (§28.1). Player flow (§1) updated. PDF: Mermaid diagrams now rendered as graphics — they appeared as code.
2.15	September 25, 2026	Frédéric Delorme	FTL cruise (§29): cruise $\times 5$ to $\times 10$ depending on mass, on warp rings — distinct from the quantum jump, which now relies on a jump generator (§28); selector with two combinable options (§27.2). §19: universe map marked implemented. §3 and screenshots updated with the generated ships. Project reorganized (§30): sources in <code>src/</code> (html, css, js, test, docs), Node.js build producing the compact single page in <code>target/</code> , 8 automated tests, reproducible screenshots and PDFs; seed settable from the URL (<code>?seed=</code>). Fixes: the title screen could come back over the game when the intro was skipped with a click and the game started quickly; the "Engine controls" panel stayed stuck on "Reactors 1-4" whatever the ship — two defects revealed by test and screenshot automation.

Version	Date	Author	Content
2.16	September 26, 2026	Frédéric Delorme	Rendering taken from the "Observation des étoiles" demo (§31): detailed planets (procedural surface, three cloud layers, scattering atmosphere, auroras), textured asteroids and moons, system star visible from afar, dynamic resolution, spacetime grid during the jump and star streaks during FTL cruise. Game-specific addition: geometric detail based on on-screen size, since the game keeps up to four systems built. 9th automated test (rendering).

Table of Contents

Part I — Current state of the simulator

- 1. Overview
- 2. Procedural universe
- 3. The ship
- 4. Flight controls and camera
- 5. Navigation — flight plan
- 6. Planetary systems
- 7. Arrival sequence: orbit, shuttles, credits
- 8. Interface (HUD)
- 9. Technical architecture
- 10. Internationalization

Part II — Remaining developments to implement

- 11. Pause mode —  implemented
- 12. HUD control bar —  implemented
- 13. Random events and repairs
- 14. Radio dialogues generated by embedded AI (Gemini Nano) —  implemented
- 15. Touch controls for mobile devices —  implemented
- 16. Port services —  implemented
- 17. Fixes and improvements (post-Tier-1 refinement)
- 18. Implementation plan
- 19. Universe map —  implemented (M key)
- 20. Reference — all keyboard shortcuts
- 21. Visual itinerary and Lagrange point —  implemented
- 22. Fuel and consumption —  implemented
- 23. Quantum propulsion and star map —  implemented
- 24. Orbital ports
- 25. Title screen: keyboard navigation and background flythrough —  implemented
- 26. Background music and volume controls —  implemented
- 27. Generated ships and selection screen —  implemented

- 28. Quantum jump v2: spacetime distortion — ☒ implemented
- 29. FTL cruise — ☒ implemented
- 30. Project organization, build and tests — ☒ implemented
- 31. Celestial rendering and propulsion effects — ☒ implemented

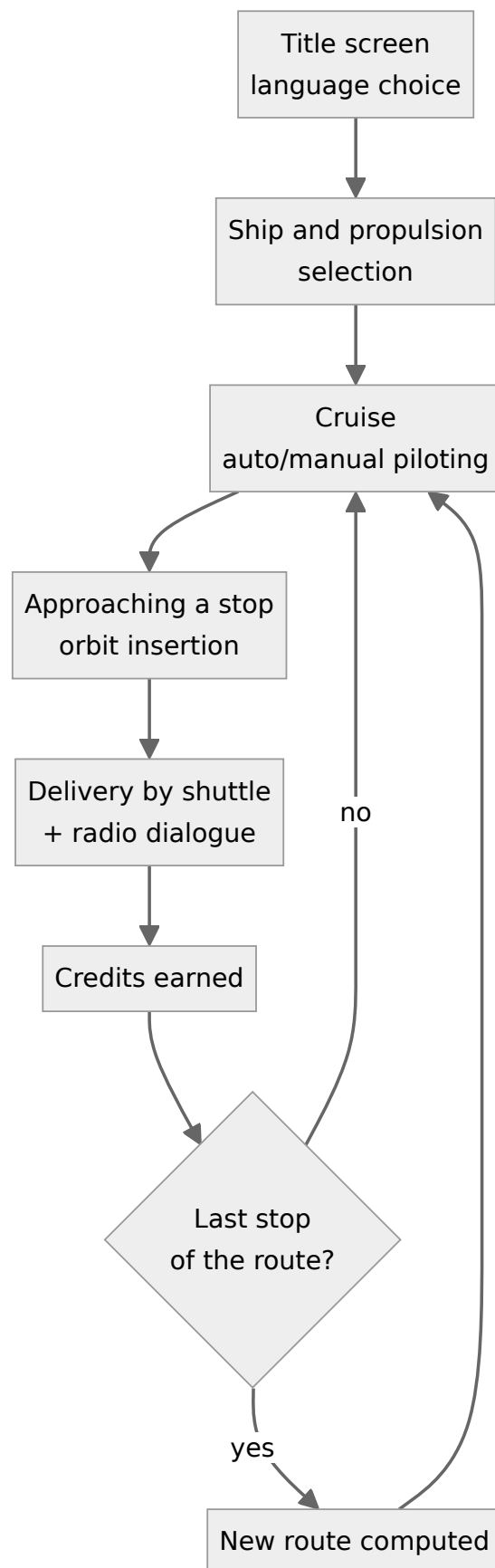
Part I – Current state of the simulator

1. Overview

Procedural space-flight simulator, shipped as **a single self-contained HTML file** (Three.js r128, WebGL, no build step). A cargo ship pilots automatically — or manually — from system to system, delivers its cargo by shuttle at each stopover, exchanges radio traffic with port control, earns credits, and continues its route through a universe generated entirely from a single seed. Interface available in French, English, German and Spanish, chosen on the title screen.



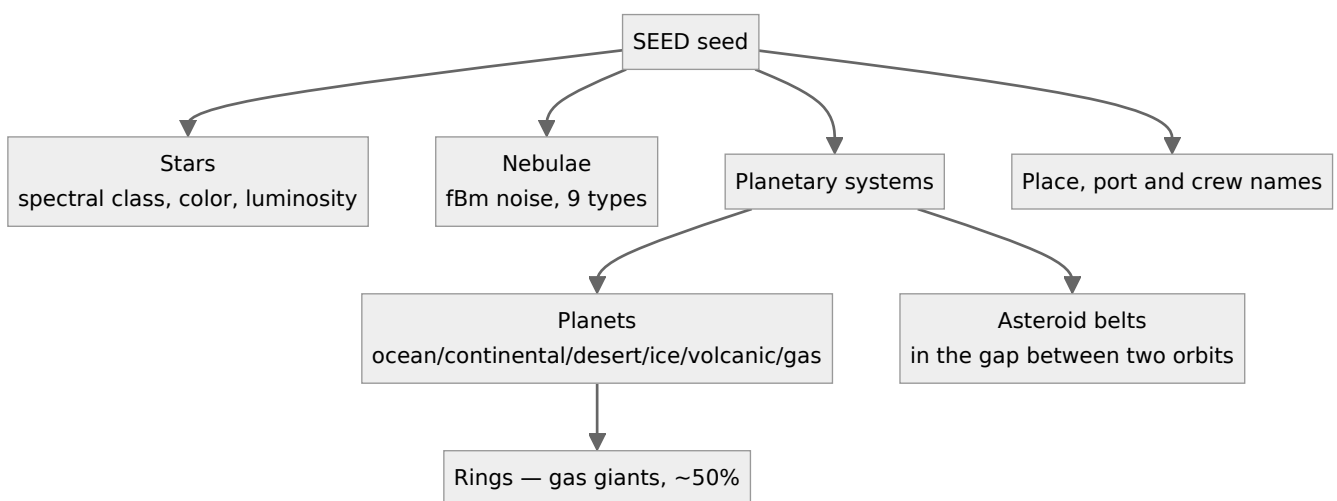




2. Procedural universe

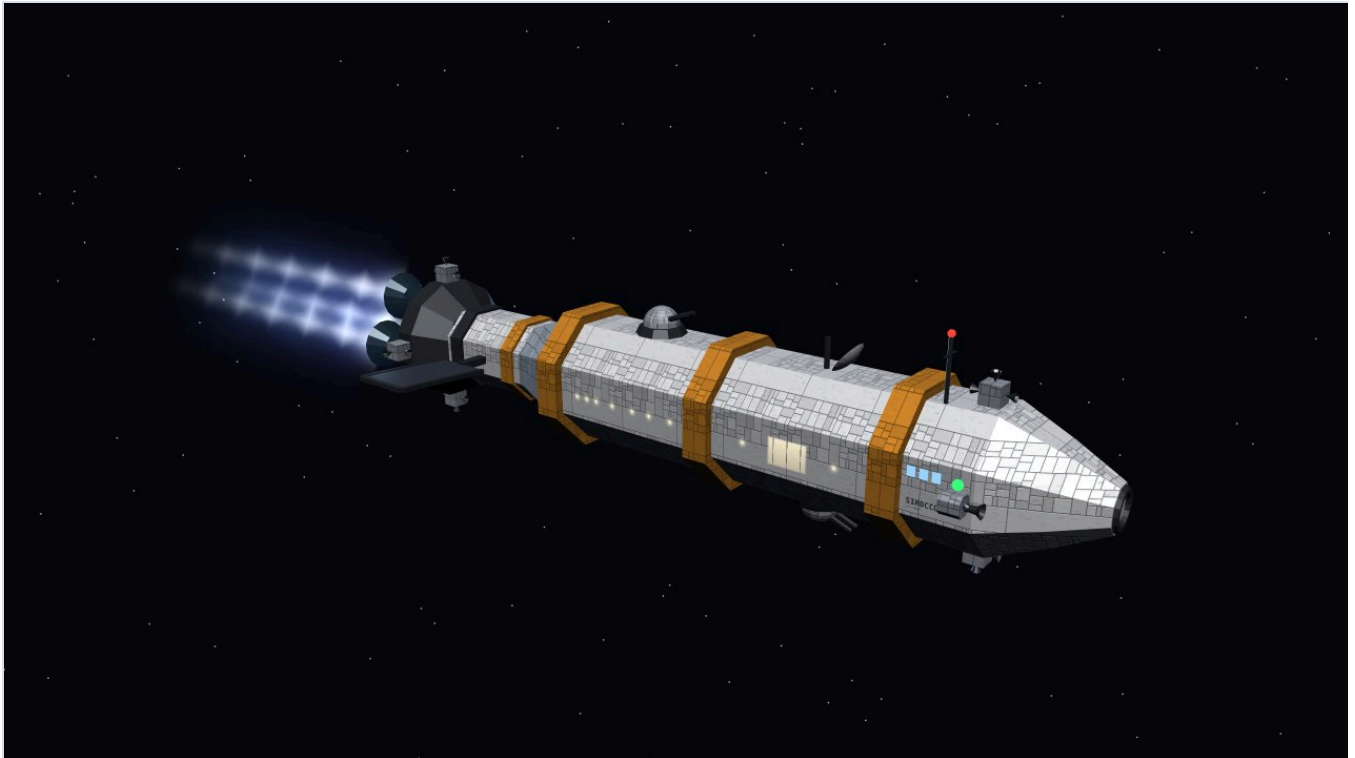
Everything — stars, nebulae, planetary systems, names — derives from a single seed (SEED) via dedicated pseudo-random generators. Replaying the same seed reproduces exactly the same universe.

- **Stars** physically modeled: spectral classes (Salpeter fractions), color by Planckian locus, luminosity/mass/radius from astrophysical laws, apparent magnitude computed from distance.
- **Nebulae** procedural (4-channel fBm noise), fading in gradually to avoid any visual "pop" effect.
- **Designation labels:** targeting circle + arrow, for stars, planets and moons — a visual vocabulary consistent across the whole HUD.



3. The ship

Since v2.13, the ship is no longer unique: the player picks it at launch among ten generated models (§27), from the 61 m light tug to the 237 m spine freighter. All follow the same chained construction — bow (bridge, painted name, side retro thrusters, RCS, lights), habitat in modules separated by collars in the livery colors, engine section (reactor, radiators, thrust plate, Epstein nozzles) —, detailed in the study "Generative construction of cargo ships". Long-haul ships can also receive warp rings (FTL cruise, §29) and a jump generator (quantum jump, §28).



(Original ship, v2.1 to v2.12.) **New in v2.1 — hull dressing.** The four materials reused throughout the ship (hull, deck, dark panels, frames) now carry a procedural, canvas-generated texture — a grid of panels with a slightly detuned tint, marked seams, scattered greebles (hatches, vents), occasional yellow/black hazard stripes. Inspired by *Homeworld*-style cargo ships, with no dependency on an external image: consistent with the rest of the file, which already generates its ring and planet textures the same way. One texture per material is enough to dress the whole silhouette; the shuttles have their own texture, generated once at load time rather than on every launch.

(Original ship, v2.1 to v2.12.) **New in v2.1 — shuttle dock.** A blue-lit bay, jutting out under the rear (propulsion) block — not under the cargo pod, where it would have ended up hidden by the bulk of the containers. Well + metal frame + emissive surface + double additive halo (tight core, wide glow) + a short-range point light that actually lights up the surrounding hull, with a gentle breathing pulse rather than a blink. Shuttles genuinely launch from this point, not from the ship's center.

Since v2.13, this dock and its loading arm are placed under the habitat module of each model — under the rear bulkhead for the warehouse freighter (§27.3).

4. Flight controls and camera

Since v2.13, the ship is no longer unique: it is chosen at launch among ten generated models (§27), and both the chase camera and avoidance radii adapt to its size.

Key	Action
↑ ↓ ← → / Z Q S D (AZERTY) / W A S D (QWERTY)	Orientation
E / R	Roll (fixed in v2.1 — the help panel mistakenly showed A / E)
SHIFT or SPACE	Thrust (boost)
Mouse drag	Fine aim
CTRL + drag	Free look
F3	Next camera mode (new in v2.1)

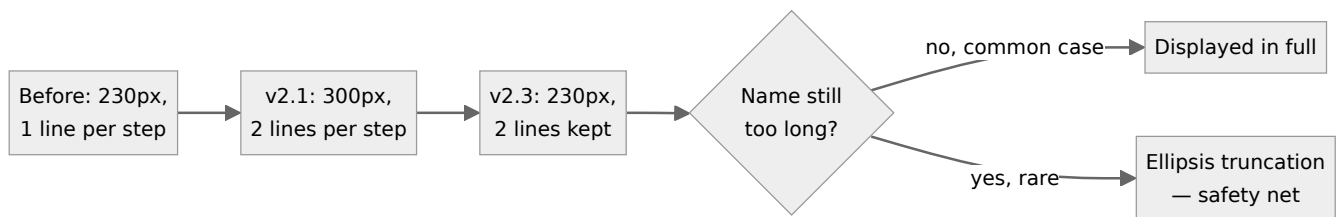
New in v2.1 — F3. Switching camera mode (standard chase → cinematic tracking shot → distant shots) is now a single press of F3, replacing the old double/triple-press-on-CTRL mechanism. The latter — sensitive to rendering conditions during its initial validation — was **removed entirely** rather than patched: CTRL regains a single role (free look while dragging, recentering on a short press), with no timer needed to distinguish one, two or three presses.

5. Navigation – flight plan



Fix v2.1 — panel width. Measured in the code: the panel was only 230px wide, with number/name/star designation on a single line — well short of what generated port names need, causing frequent truncation. Two combined adjustments: moderate widening to 300px, **and** a switch to a two-line-per-step layout (name at full width, star designation below, dimmed). Truncation remains as a safety net for the rare names still too long.

Revision v2.3 — back to 230px. Explicit request to narrow the panel. Reverting to the original width did **not** reintroduce the truncation problem v2.1 had fixed: it was the switch to the two-line layout, kept intact, that did the real work (an overly long name gets its own line rather than being squeezed next to the number and designation). Width alone was only one of the two levers.



The route is planned via a Catmull-Rom curve passing through each star and then each target planet, with two safety passes that steer clear of any planet — targeted or not — that would otherwise end up too close to the path (verified collision-free across 150 randomly generated routes).

Bug fixed — the ship seemed to advance jerkily. Progress along the curve (ROUTE.s, which feeds the autopilot's heading calculation, §4) was recomputed every frame via a nearest-point search over 24 steps spread across 420u — a step of about 17.5u, far coarser than the distance covered per frame at cruise (~0.9u at 56u/s, 60fps). ROUTE.s therefore stayed frozen for about twenty frames before jumping a full step at once, shifting the targeted heading in jerks rather than continuously — perceived as jerky motion. A first densification (240 steps, ~1.75u) still remained perceptible: the delta still alternated between 0 and 1.75u from one frame to the next. Replaced with a **two-pass** search — a coarse one (24 steps, finds the right neighborhood) then a fine one (32 steps narrowed to ± one coarse step around that neighborhood, ~1.1u final resolution) — for 56 total iterations, cheaper than the first attempt's 240 steps, with a result that never freezes again: verified with not a single stalled frame out of 100, versus 92 before the fix.



6. Planetary systems



- 2 to 3 planets per system, one habitable (space port).
- Orbital spacing in geometric growth (Titius-Bode spirit), with a mathematical guarantee of zero overlap regardless of orbital angle.
- Rings on ~50% of gas giants; asteroid fields placed in the **gap** between two orbits, never on a planet's orbit.

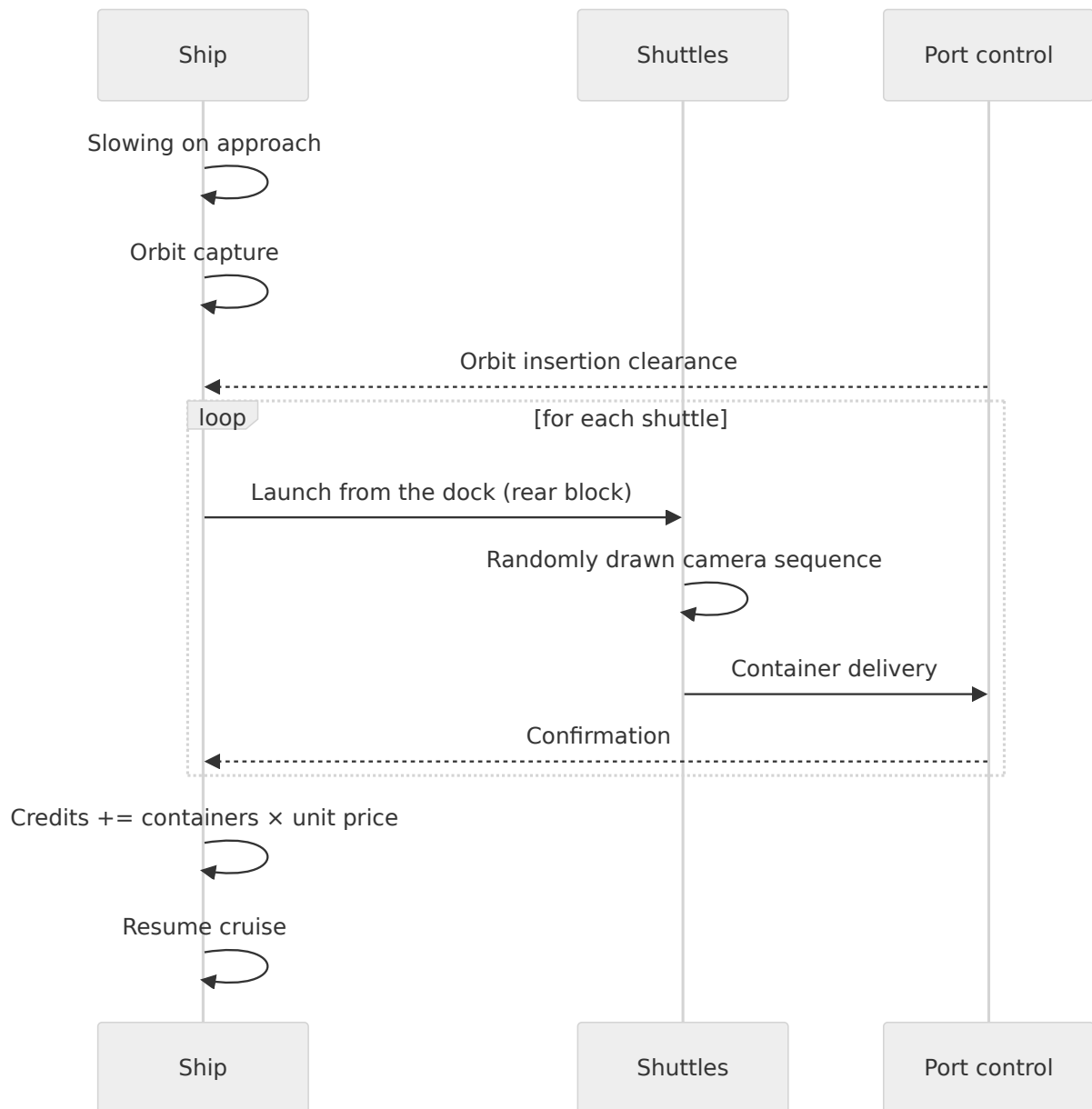


7. Arrival sequence: orbit, shuttles, credits

At every stop (not just the last), the ship slows on approach, enters orbit, and delivers its cargo.



Shuttle in flight with its thruster trail (§17.2) aligned to its actual direction of travel — not to its nose orientation, which can diverge during the curved approach phase. Port services panel (§16) open in the foreground.



New in v2.1 — shuttle staging. A catalogue of four shot sequences, drawn at random on each launch (seeded, so reproducible for a given seed):

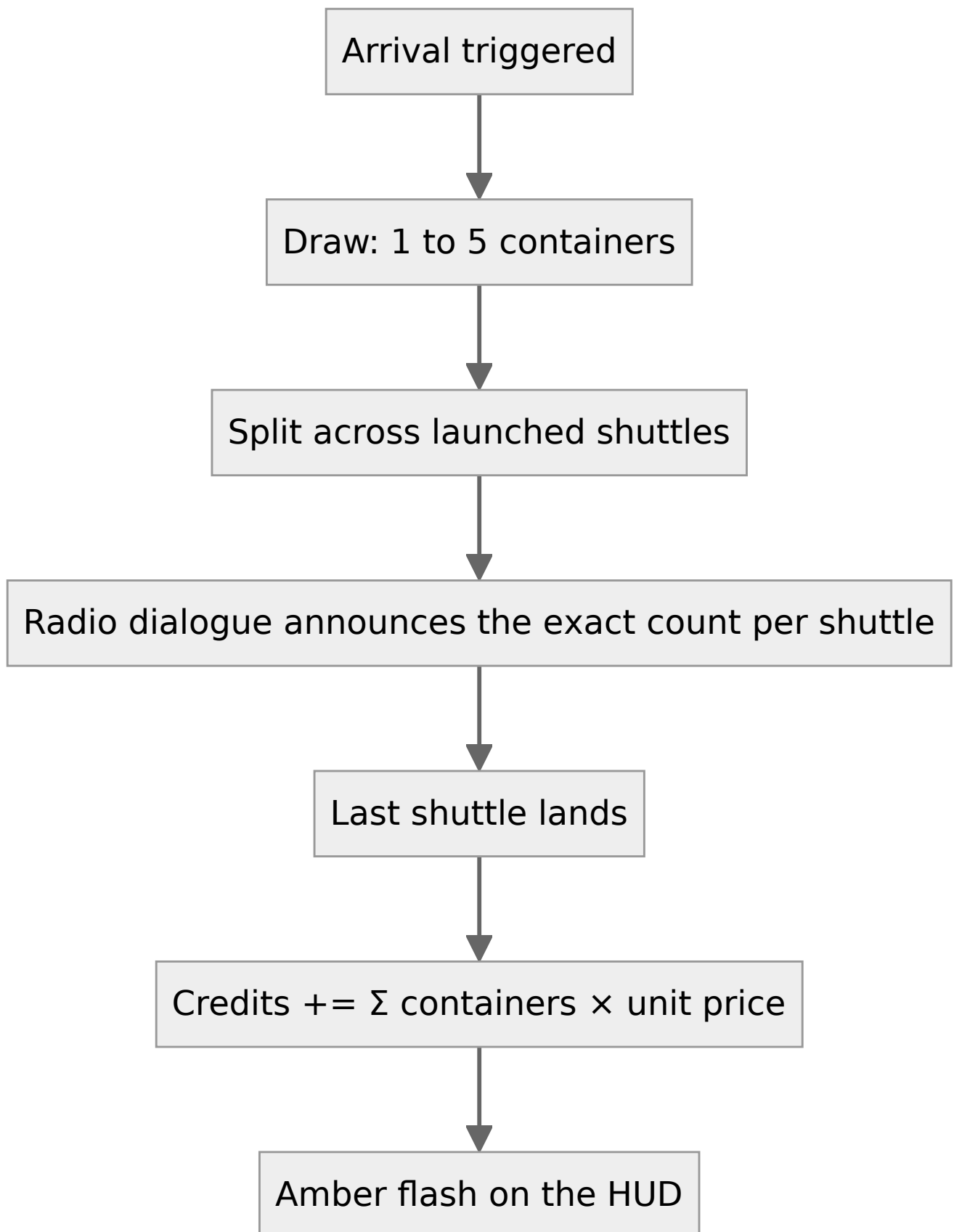
Sequence	Initial close-up	Then
Close lateral	Dock seen from the side, low angle	Hard cut to a distant tracking shot
Head-on	The shuttle comes straight at the camera	Wide lateral tracking shot, the port enters frame
Continuous tracking	Dock in close-up	Gradual pull-back, no cut at all
Over the shoulder	Dock in close-up	Fixed shot, locked to the ship

The hard cut (for the first three sequences) was verified by direct instrumentation: a camera jump of more than 120 units in a single frame, exactly at the intended moment — not a gradual fade.

New in v2.1 — credits. 1 to 5 containers drawn per stop, split across the shuttles actually launched — and this same split feeds the radio dialogue, so the crew never announces a number unrelated to what's actually credited. Payout happens the moment the last container lands. Starting balance of 10,000 credits, shown in the top bar to the left of SEED.



Stacking order reversed in v2.3 (§17.1): port services (§16) now sit between "nearest object" and the radio channel, rather than the reverse — a dynamic overlap had been spotted in the old order.



The radio channel combines text dialogue and speech synthesis (voice randomly drawn male/female for each speaker), with synthesized beeps as each message appears.



8. Interface (HUD)



Panels: Speed/Sector/Heading, Flight plan, Nearest object, Engine temperature, Propulsion, Engine commands, Radio channel (toggle TAB). Keyboard help (H) kept up to date with all current shortcuts, including F3/F10.

9. Technical architecture

9.1 The seed generator

The whole universe — stars, nebulae, planetary systems, place names — derives from a single starting value, the seed (SEED), through a two-stage mechanism: a string hash (xmur3) that turns a **text tag** into a 32-bit integer, followed by a fast pseudo-random generator (mulberry32) that turns that integer into a reproducible stream of numbers.

Why two stages, not a single global stream. A single generator advanced sequentially would make every draw depend on everything drawn before it — the same star cell would yield a different star depending on whether it's reached directly or after a long detour, since the generator wouldn't have consumed the same number of values along the way. The game needs the opposite: a planet, a star or a name must always be **the same**, regardless of the order in which the player explores the universe — a condition essential for sliding-window generation (§2), which never builds the whole universe at once.

The fix: every generated element gets its own unique, descriptive text tag (for example SEED+' :route:3' for a route's 4th step, or a combination of cell coordinates for a star), which feeds an **independent, replayable-at-will** number stream:

```
rngFor(tag) = mulberry32( xmur3(tag)() )
```

First stage — `xmur3`, an avalanche hash. A function borrowed from the MurmurHash3 family, it turns a character string into a 32-bit integer by mixing in each character in turn:

```
h0 = 1779033703 ⊕ length(string)
hi = rot13( (hi-1 ⊕ code(ci)) × 3432918353 mod 232 )
```

where `rot13` is a 13-bit bitwise rotation and `×` a 32-bit multiplication (`Math.imul`). A finishing pass — three rounds of shift/XOR/multiply by odd constants — guarantees that changing a single character of the tag, even a single bit, flips on average half the bits of the result (the avalanche property): a necessary condition for two neighboring tags (`' :route:3'` and `' :route:4'`) to produce seeds with no perceptible correlation.

Second stage — `mulberry32`, the actual generator. Starting from the integer produced by `xmur3`, each call advances the state by a simple increment:

```
an+1 = an + 0x6D2B79F5 (mod 232)
```

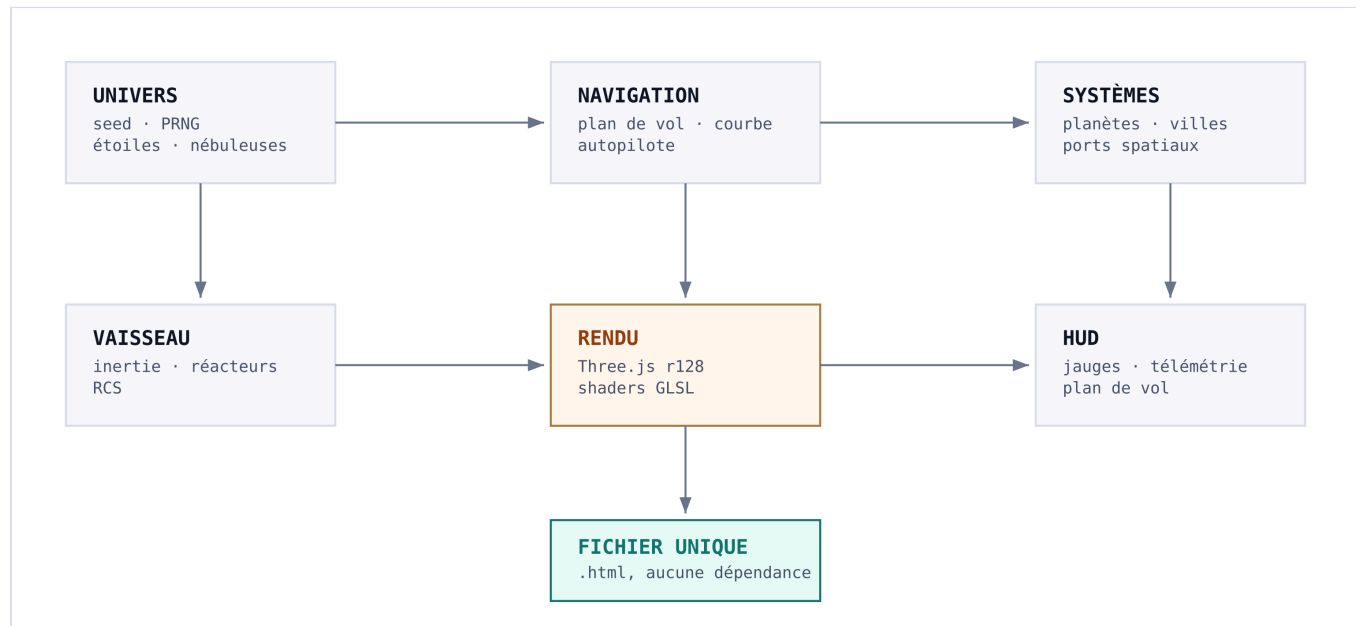
then derives a floating-point number through a combination of shifts and multiplications (the "scrambling" step):

```
t = (an+1 ⊕ (an+1 >> 15)) × (an+1 | 1)
t = (t + ((t ⊕ (t >> 7)) × (t | 61))) ⊕ t
output = (t ⊕ (t >> 14)) / 232
```

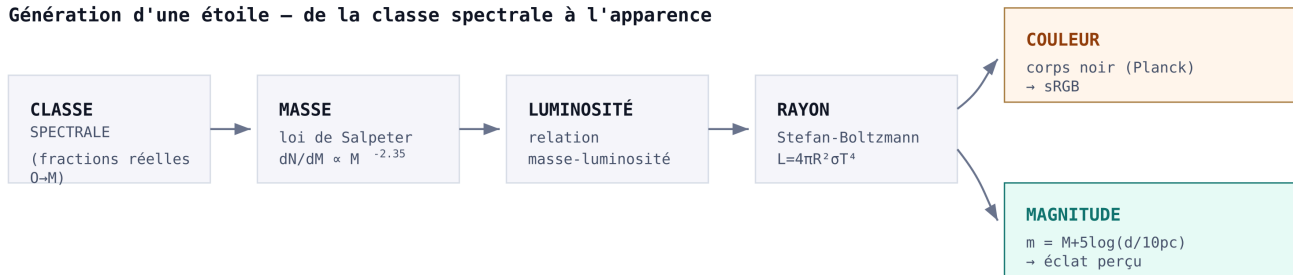
The increment alone would already be enough to cycle through all 2³² possible values without ever repeating before the end of the period — 0x6D2B79F5 is odd, which mathematically guarantees a full cycle. The scrambling exists purely to distribute those values statistically, not to avoid repeats. Mulberry32 is deliberately chosen for its simplicity (a handful of integer operations, no dependencies) rather than for cryptographic quality that would be wasted here: good enough that no visual pattern is perceptible in a star field or a planet distribution, nowhere near enough for a use case that needs genuine unpredictability.

Practical consequence. Replaying the same seed (`SEED`) reproduces exactly the same universe, down to the last pebble in an asteroid field — the seed itself is just one more ingredient in the tags fed to `rngFor`, never consumed directly.

9.2 Architecture diagrams

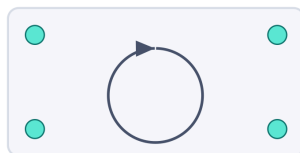


Génération d'une étoile – de la classe spectrale à l'apparence



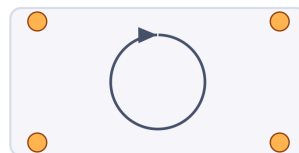
Disposition des 16 propulseurs d'attitude (RCS), par axe de couple

VUE DE DESSUS – LACET



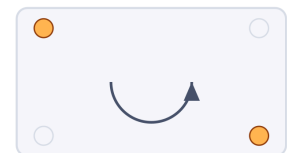
4 tuyères · couple autour de Y

VUE DE CÔTÉ – TANGAGE



8 tuyères (×2 par coin) · couple X

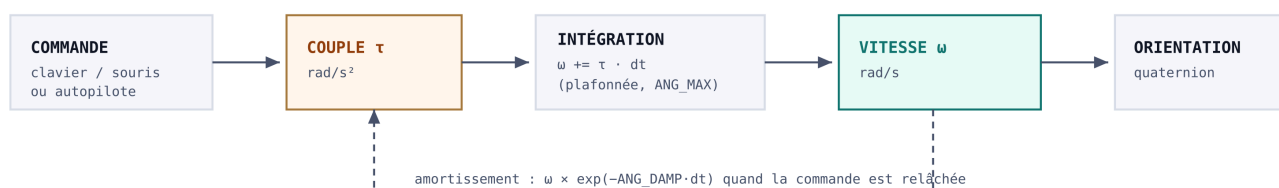
VUE ARRIÈRE – ROULIS



4 tuyères · couple Z

Chaque tuyère émet un jet conique orienté vers l'extérieur ; les paires opposées s'allument ensemble pour produire un COUPLE PUR, sans pousser le vaisseau hors de sa trajectoire. Numérotées 1 à 16 dans le panneau de jauges (1-4 lacet, 5-12 tangage, 13-16 roulis).

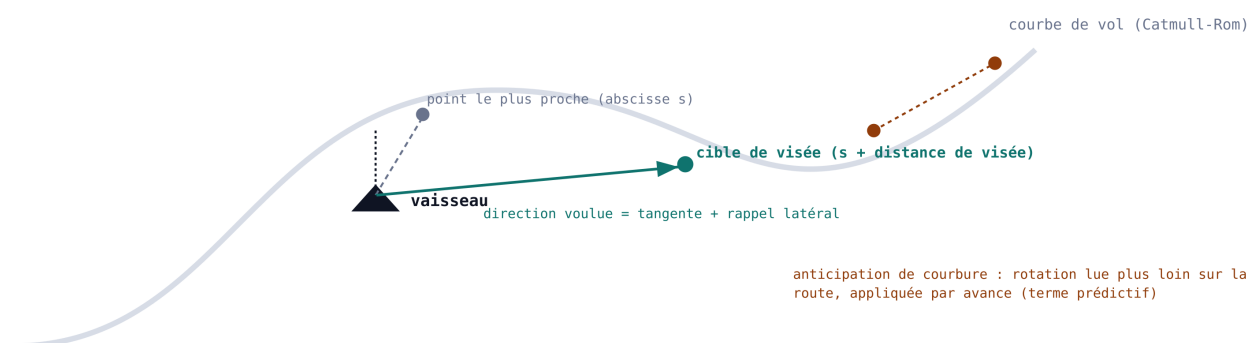
Boucle d'inertie de rotation – du couple à l'orientation



Sans cet amortissement, en l'absence de frottement, la rotation ne s'arrêterait jamais.

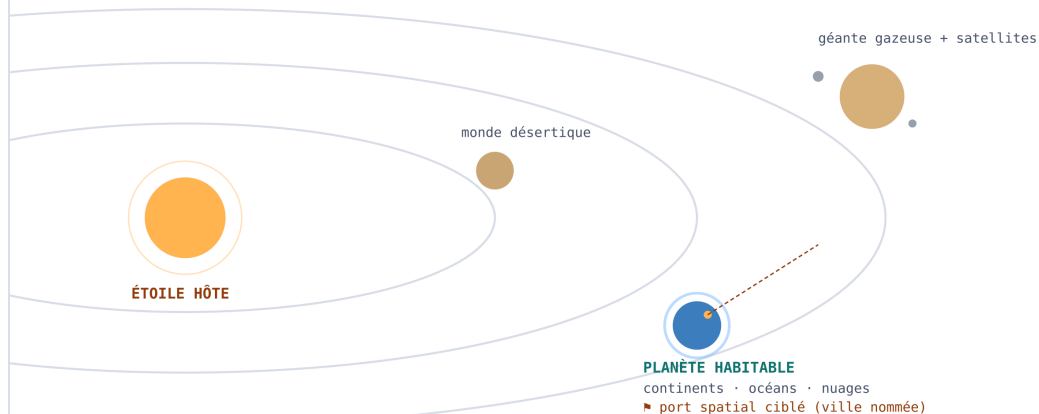
Un cargo de plusieurs centaines de tonnes ne réagit pas non plus instantanément à une commande : c'est ce délai d'intégration qui donne la sensation de masse.

Autopilote – suivi de courbe par champ de vecteurs



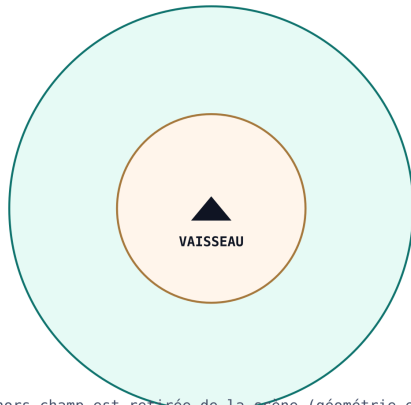
La consigne de vitesse angulaire (boucle externe) combine la correction de cap et l'anticipation ; une boucle interne l'asservit ensuite en couple – sur les DEUX composantes du vecteur, pas sa seule projection.

Structure d'un système planétaire généré (2 à 3 planètes par étoile)



Les planètes NE révolutionnent PAS autour de l'étoile : seule leur rotation propre (spin) est animée, pour que leur centre reste exactement là où la route a été tracée.

Fenêtres de construction – seul ce qui compte est réellement en scène



■ CHAMP D'ÉTOILES NOMMÉES

rayon = 3 cellules (≈ 570 u) autour du vaisseau
taille et éclat suivent la loi en carré inverse

■ CHAMP DE NÉBULEUSES

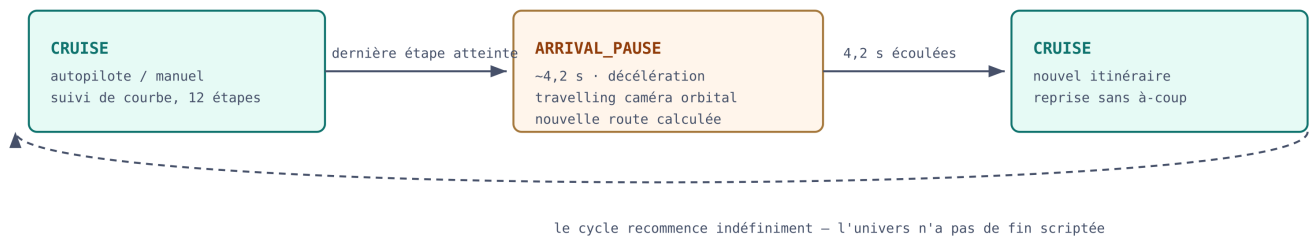
rayon = 1 cellule (≈ 1600 u), amas de bouffées
instanciées, coût GPU maîtrisé

■ SYSTÈMES PLANÉTAIRES

construits pour l'étape EN COURS + la SUIVANTE
seulement (fenêtre sur l'index de route, pas la
distance) – démontés dès l'étape franchie

Chaque cellule hors champ est retirée de la scène (géométrie et matériaux disposés) dès qu'elle sort de la fenêtre suivie – c'est ce qui permet un univers procédural sans limite de taille.

Cycle d'un itinéraire – de la croisière à la relance automatique



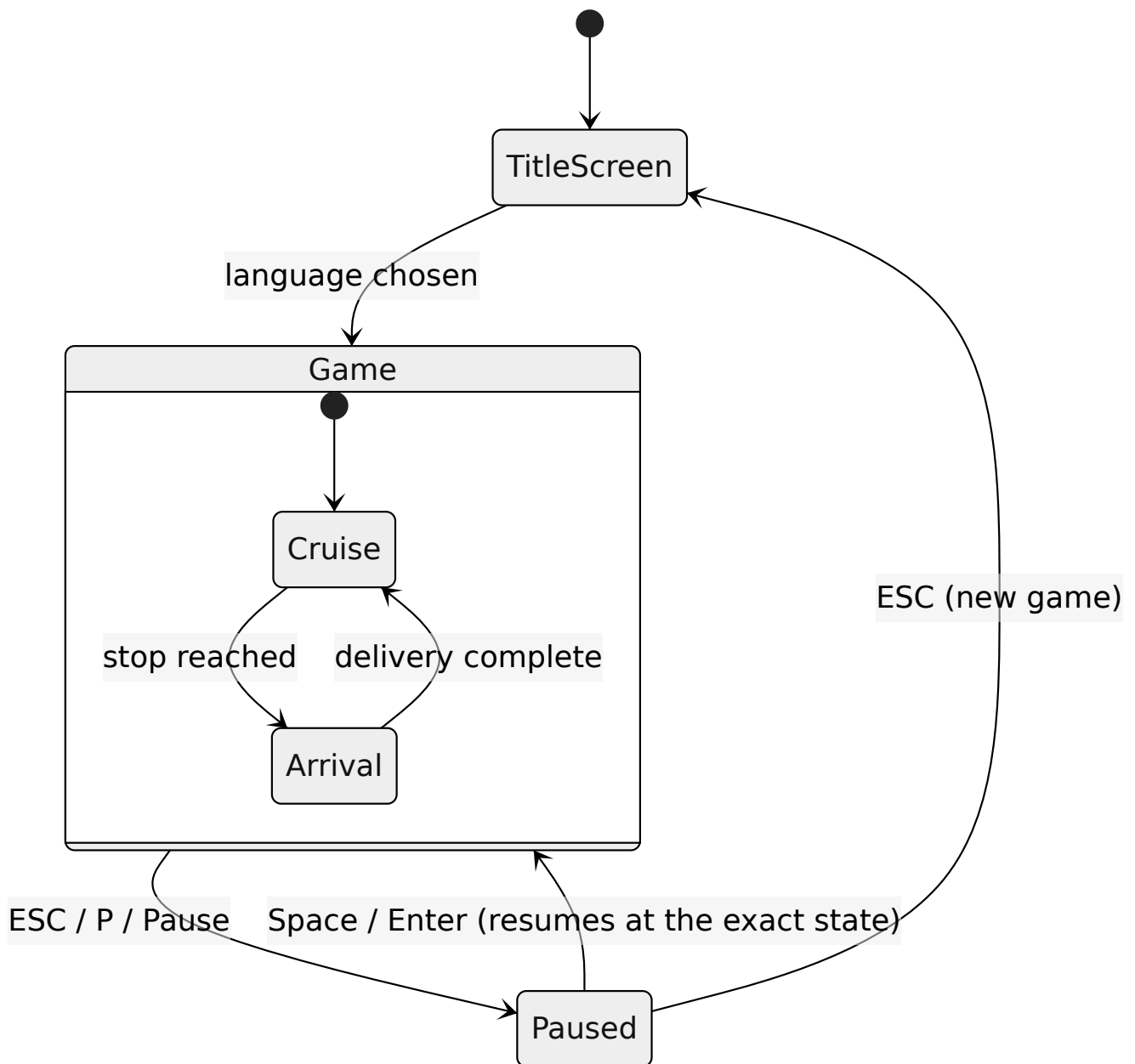
10. Internationalization

French, English, German, Spanish — an I18N dictionary covering HUD labels, radio dialogue templates (parameterized, not hardcoded strings), and the speech-synthesis language. Chosen once on the title screen, applied for the whole session.

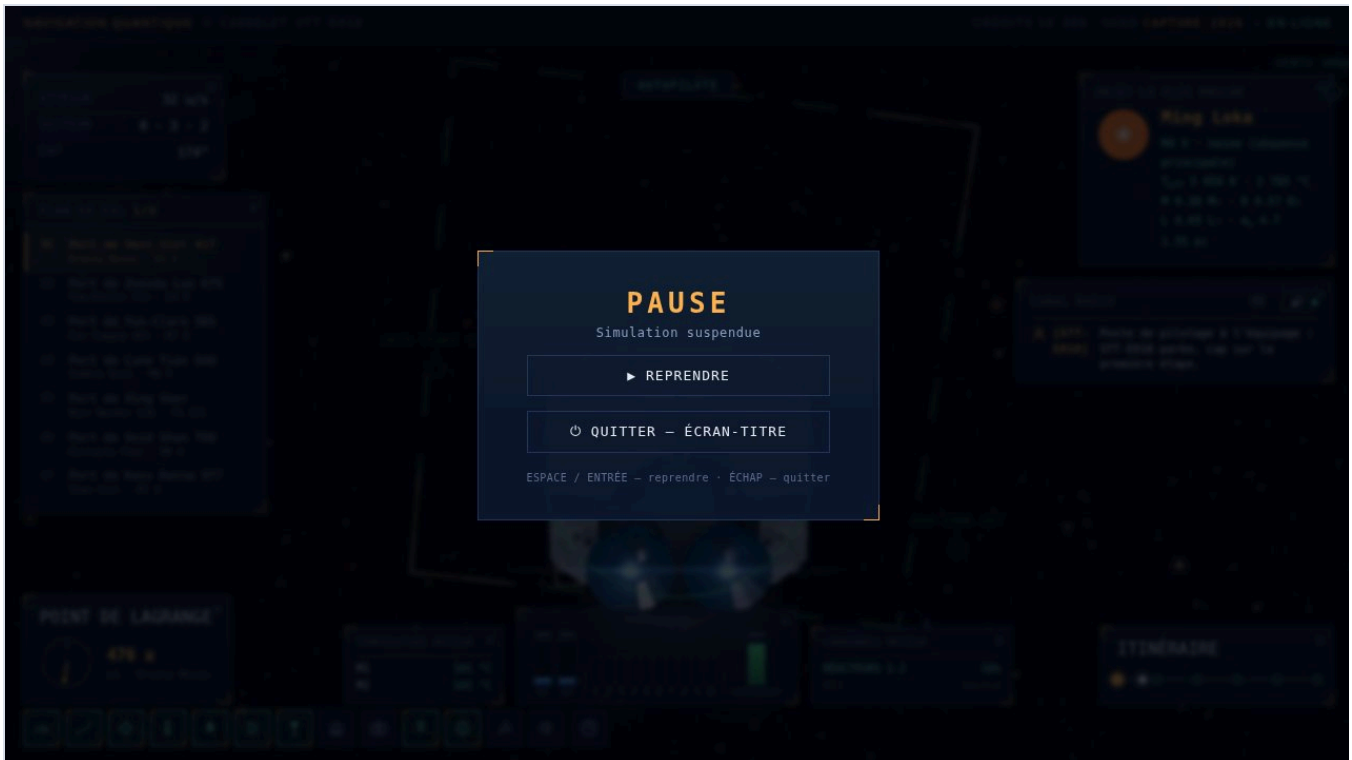
Part II – Remaining developments to implement

The following efforts remain at the specification stage: they are **not** present in today's shipped file.

11. Pause mode – implemented



ESC, P or PAUSE freezes the simulation entirely (route, rotation, temperatures, shuttles) and suspends speech synthesis without cancelling it (`speechSynthesis.pause()`, not `cancel()`). SPACE/ENTER resumes exactly where the game left off; ESC from pause returns to the title screen.



Differences from the previous version of this spec:

- **No renaming** of `ARRIVAL_PAUSE` / `paused` : the confusion risk flagged was real but purely documentary (distinct JS scopes, no actual conflict) — a new, unambiguously named flag (`gamePaused`) was enough, at far lower cost and regression risk than a rename touching the whole file.
- **ESC no longer ever acts as a maneuver skip** (a role it used to share with `SPACE/ENTER`) — removed from that role to eliminate any conflict with pause, which becomes its sole function, in every game phase without exception.
- **No confirmation before quitting**: `ESC` while paused reloads the page immediately (a deliberate choice — see below).
- Returning to the title screen is a **full page reload** (`location.reload()`), rather than manually resetting dozens of interdependent global variables (`ROUTE`, `orbitState`, `credits`, `ship` orientation, propulsion upgrade level...) — this guarantees a clean state and a fresh seed (`SEED` derives from `Date.now()`) with no risk of missing one. Accepted consequence: the upgrade level bought at port services (§16) also resets to zero, consistent with "new game."

12. HUD control bar – implemented

An SVG icon bar, generated dynamically from a single list (`HUD_BAR_ITEMS`) — desktop and touch share exactly the same toggle mechanism, the same icons and the same activation logic, only the position differs.

- **Desktop**: fixed at the bottom left, always visible, as originally planned.
- **Touch**: also at the bottom, centered — flight controls (joystick, roll, boost, free look) are shifted up to make room for it (§15). *Briefly moved to the top for lack of space, moved back down on*

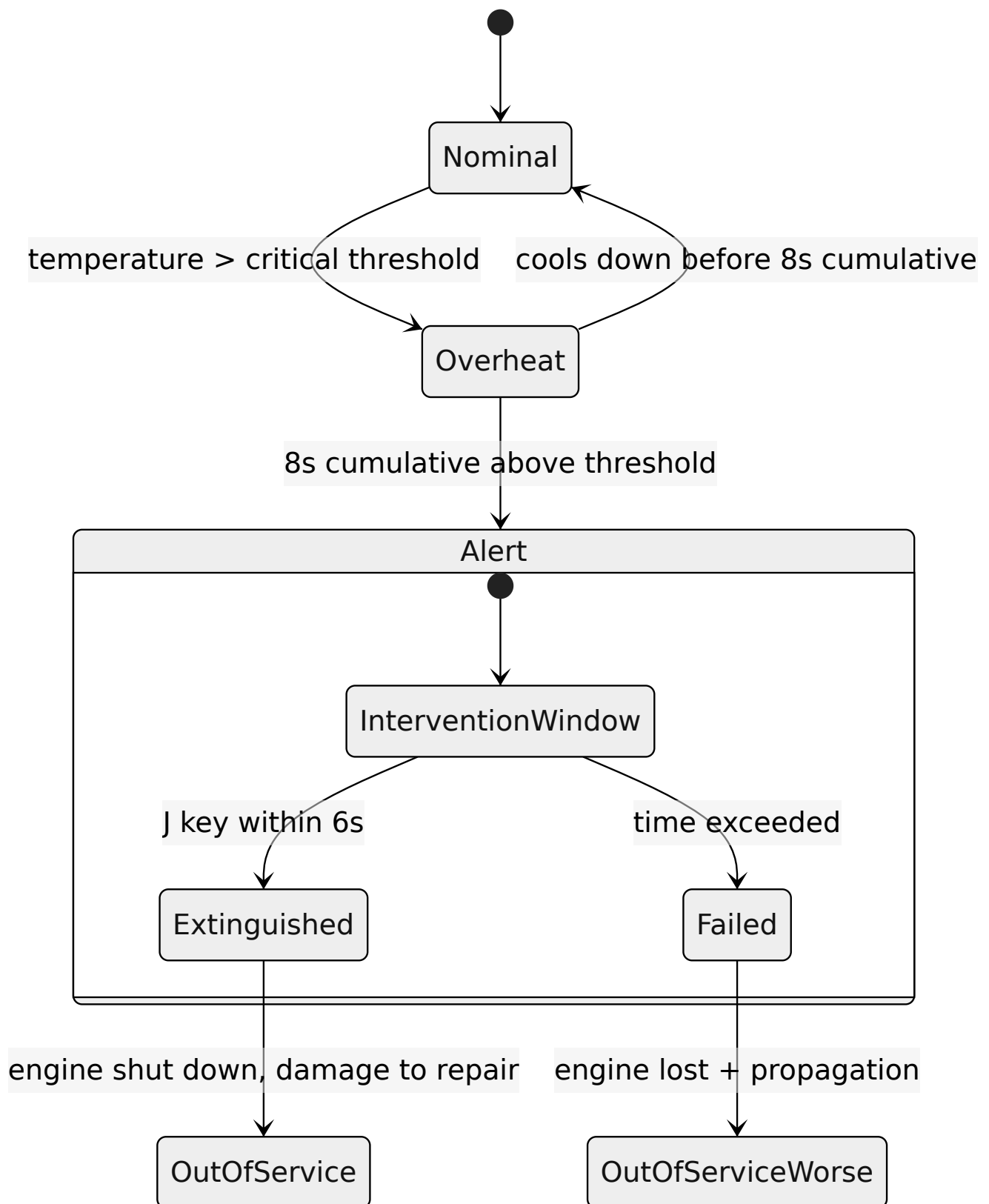
explicit request — see §17.3. Fully replaces the old touch Ξ menu (text dropdown list), now unified with this same bar.

- **12 buttons, not 8**: the six info panels, the radio channel, the camera (F3 originally) — plus **port services** (§16, icon greyed out/inert outside its activation window), **help** (§17.3), and, since v2.4, the **itinerary** and **Lagrange point** (§21).
- **Shortcuts F1 to F9**, one per button in display order, except help (H), itinerary (I) and Lagrange point (L), which keep fixed keys rather than an automatic F-slot. Replaces the old dedicated assignment of F3 to the camera (now F9, like the others). See §20 for detail.

Implementation note (v2.4): these F1-F9 shortcuts are dispatched by POSITION in the HUD_BAR_ITEMS array, not by matching each entry's hotkey property — an addition inserted before radio / port / camera would silently shift F7-F9 onto the wrong actions. For this reason, fixed-key entries (H , then I / L) are always added *after* the first nine.

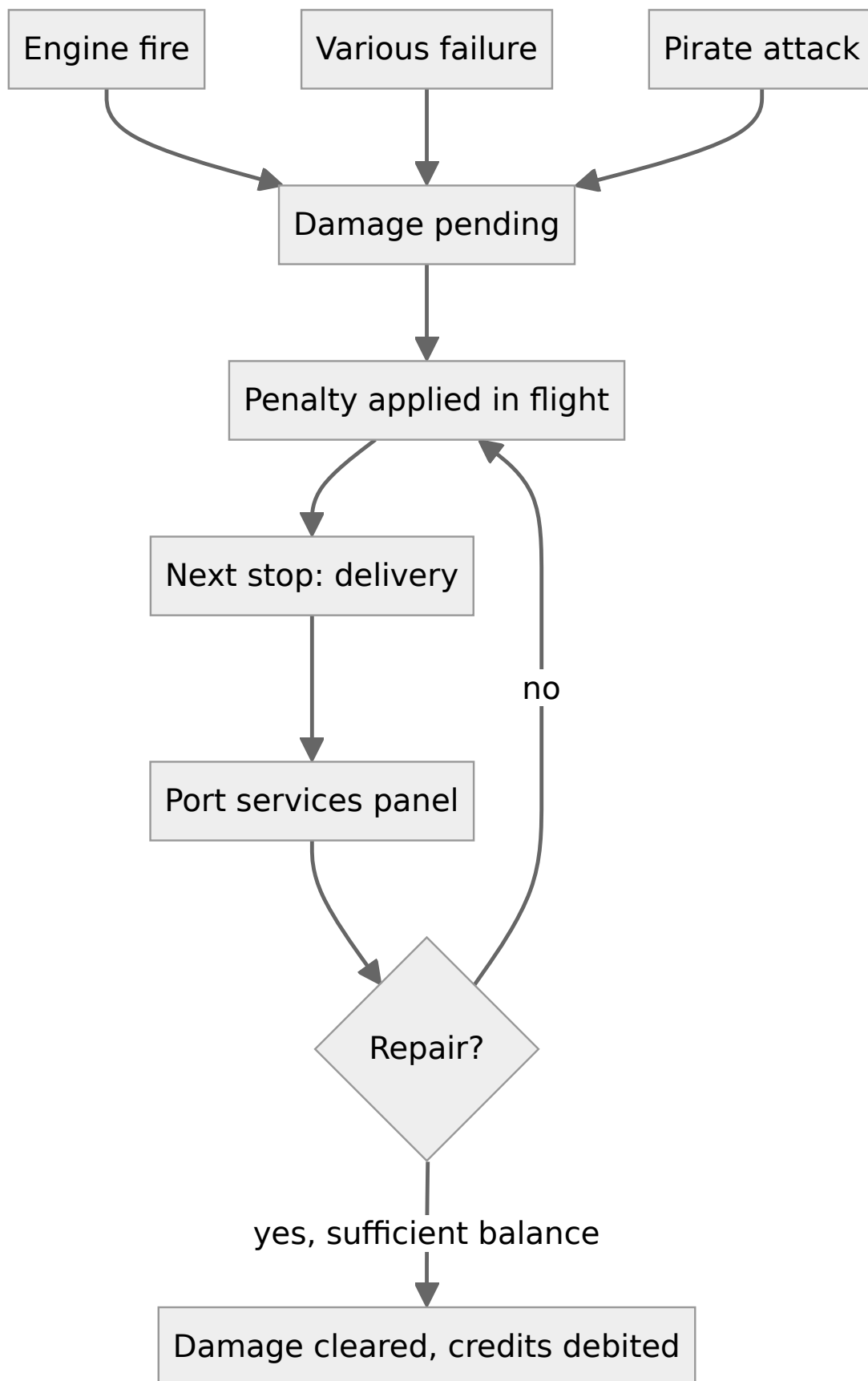
- Unified visibility mechanism: a single class, .panel-hidden , added to hide a panel — works identically on click, from the keyboard (F1 - F9) and via each panel's close button (§17.2). In touch portrait, the six panels other than the radio channel are pre-hidden at startup (no room to keep them all open at once); in landscape and on desktop, they start visible.

13. Random events and repairs



Three families of incidents sharing a common downstream path — damage to repair at the next stopover:

- **Engine fire** (see above): intervention via the **J** key (moved from **I**, taken by the itinerary — §20, §21), 6-second window.
- **Various failures**: radar, RCS thrusters, autopilot — each with its own in-flight penalty.
- **Pirate attacks**: three threat profiles (shuttle, interceptor, cruiser), branching dialogue (pay / resist / flee), border color coded by event type (amber = failure, red = hostile threat).



Repairs are offered in the same "port services" panel that already handles spending credits — the two mechanics naturally converge at the same point in the game loop.

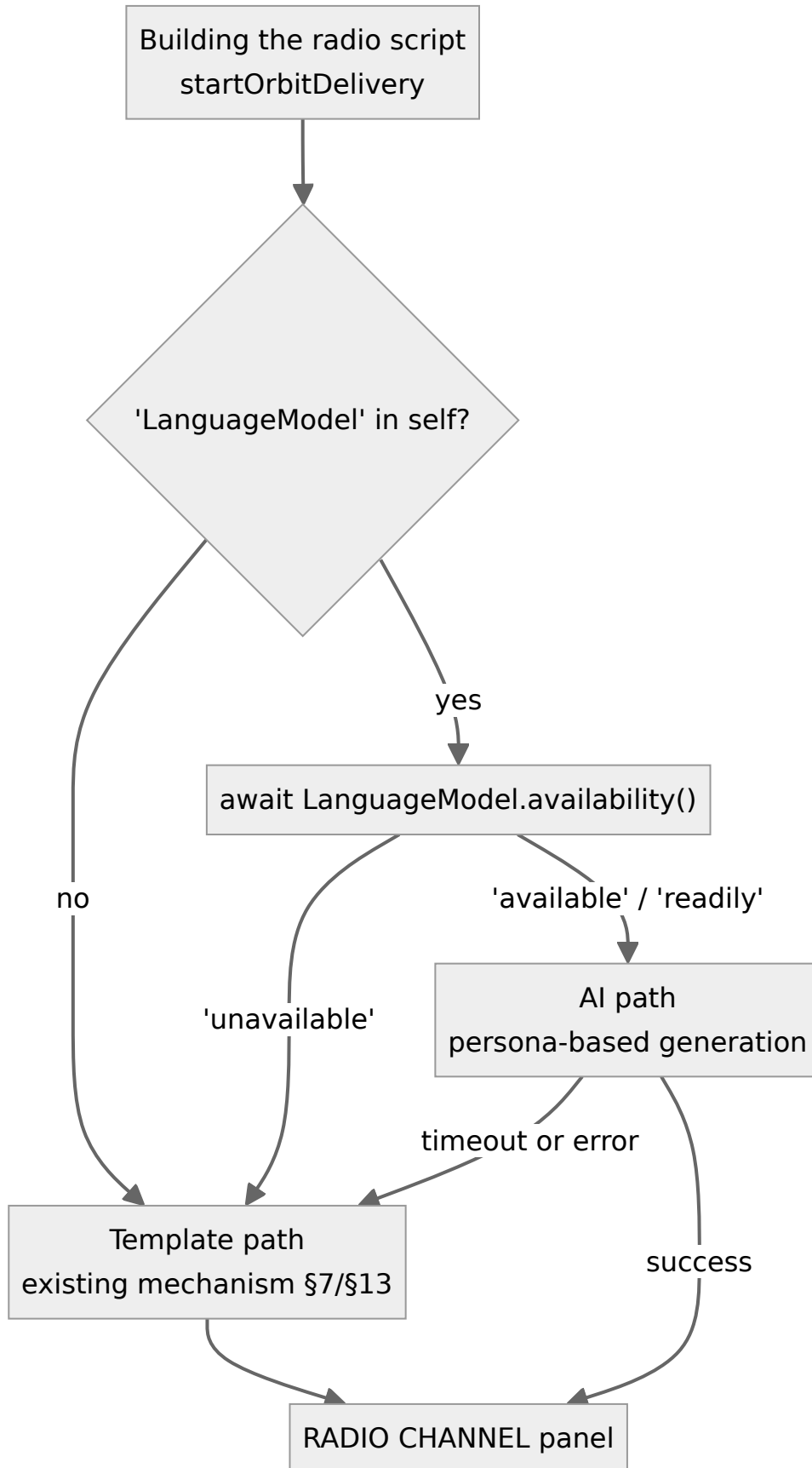
14. Radio dialogues generated by embedded AI (Gemini Nano) – implemented

Principle: where possible, replace template-generated radio dialogues (§7, §13) with dynamic generation via the LLM embedded in Chrome (Gemini Nano, exposed through the *Prompt API*), with automatic, transparent fallback to the existing mechanism when that LLM isn't available.

This choice does not betray the project's "zero external dependency" philosophy:

Gemini Nano runs **locally in the browser** — no network call, no API key, exactly the same principle as `SpeechSynthesis`, already used for voice (§7). It's not a cloud service, it's a browser capability.

14.1 Detection and fallback – implemented



Minor deviation from the initial proposal: the *Prompt API* changed its naming between Chrome versions ('readily' / 'after-download' / 'no' in early previews,

'available'/'downloadable'/'downloading'/'unavailable' afterward). The implementation treats both forms as equivalent rather than locking onto a single naming scheme, to stay correct if the API changes again.

Template text is **never cleared before a confirmed replacement arrives**: the template-generated script (§7) stays in memory for each line and serves directly as the fallback, with no separate backup code to maintain.

14.2 Personas – one identity per speaker – 2 of 4 implemented

Each speaker gets its own personality profile (tone, vocabulary register), supplied as a system instruction to its own `LanguageModel` session — rather than a single generic model called repeatedly.

Speaker	Register	Status
Cargo captain	Direct, professional, short sentences	✓ implemented
Port control	Formal, measured pace, set phrases	✓ implemented
Shuttle pilots	Casual, trade slang, chattier	pending — no dedicated shuttle radio channel in the current code
(pending) Pirate captain	Threatening or mocking depending on the threat profile (§13.3)	pending — §13.3 not implemented

Each session is created once and reused for the whole game (not one session per message): the personality profile is fixed, only the context passed in the user prompt changes from one message to the next.

14.3 Usage contexts

Context	Status	Related section
Container delivery	✓ implemented	§7
Failures and damage	pending	§13.1, §13.2
Pirate attacks	pending	§13.3

14.4 Tension with determinism – settled: the "accept the exception" option

Of the three options laid out in the previous version of this document, the implementation keeps the first: **dialogue remains the only non-reproducible part of the universe**, accepted as having no effect on gameplay (decorative, not mechanical). No per-seed cache was added — this choice may be revisited if a seeded "challenge mode" (option 3) ever comes about.

14.5 Performance and latency – implemented, with an added safeguard

Generation starts as soon as the radio script is built (`startOrbitDelivery`), for all 12 messages **in parallel**, well before the first one is due to display. A timeout (2s, via `Promise.race`) or an error leaves the template text in place for THAT specific message, without abandoning the AI attempt for the other messages in the same exchange.

Addition not planned in the previous version: a generation id (`orbitState.radioGen`), incremented on every new stopover, prevents a late response from an already-finished or superseded stopover from overwriting the script of the current one — necessary as soon as deliveries chain faster than the 2s maximum delay.

15. Touch controls for mobile devices – implemented

The simulator relied entirely on keyboard + mouse (§4). A full set of touch controls has been added, with a panel layout that adapts to a smaller screen and its orientation.

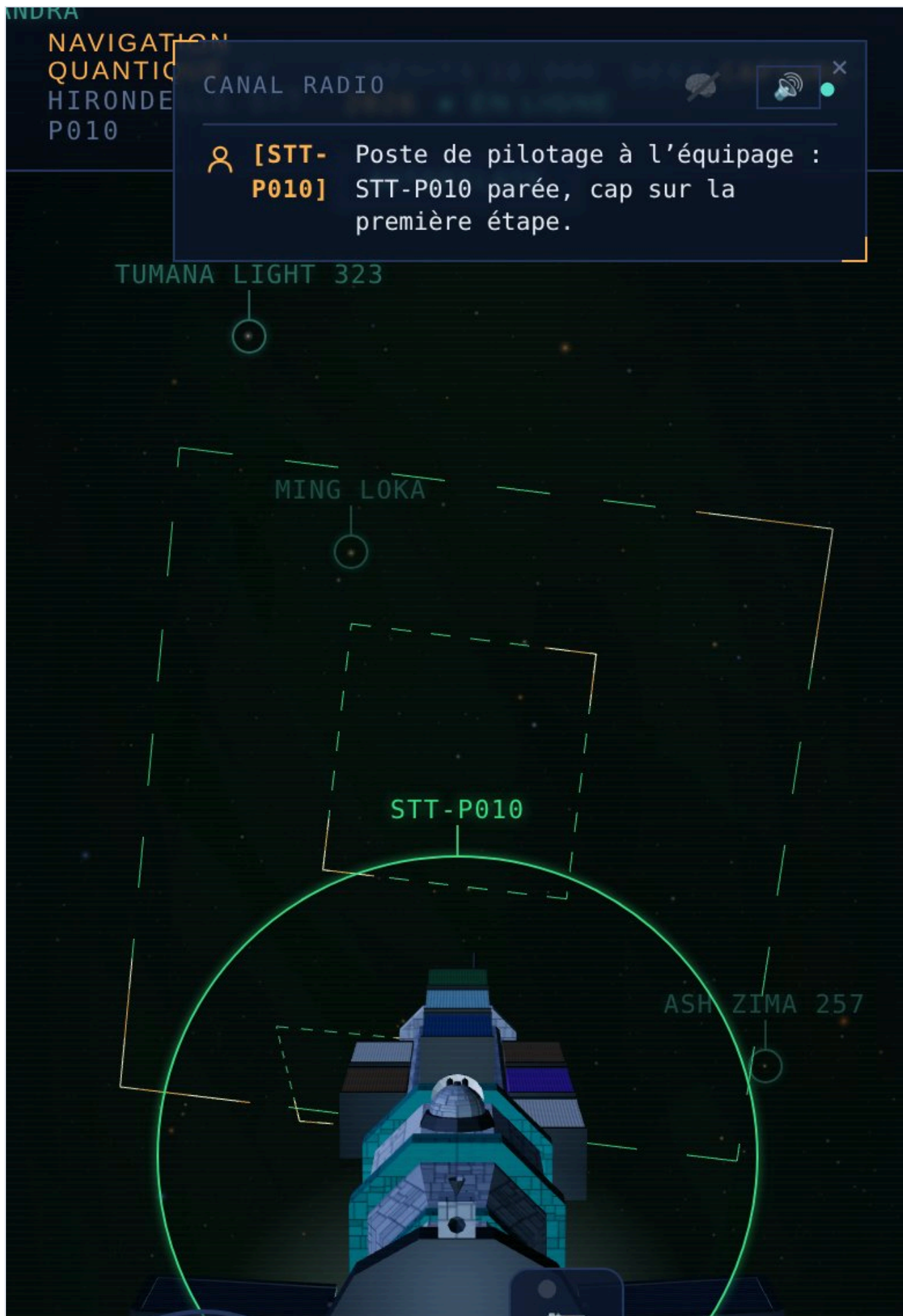
15.1 Guiding principles – confirmed during implementation

- Detection by **pointer capability** (`matchMedia('(pointer: coarse)')`), not screen size — a laptop in a small window doesn't trigger touch controls.
- Touch buttons drive **exactly the same variables** as the keyboard (`keys['ArrowUp']`, `keys['KeyE']`, etc.): no duplicated flight logic, `updateFlight()` sees no difference between the two control methods.
- **Drag a finger = look around**, no modifier to hold — implemented by branching on `e.pointerType === 'touch'` in the existing drag handler, alongside desktop CTRL + mouse (unchanged).

15.2 Portrait layout (smartphone) – updated: unified with §12

Touch zone	Keyboard/mouse equivalent	Status
Virtual joystick (bottom left)	Arrows / ZQSD — pitch and yaw	
↻ / ↺ buttons	E / R — roll	
BOOST button, held	SHIFT / SPACE — thrust	
👁 button, held	CTRL + drag — free look (§17.3: <i>no longer automatic</i>)	
Drag a finger on the 3D view	Mouse drag — fine aim → free look ONLY while 👁 is held	
HUD icon bar (bottom, centered)	F1 - F9 + H — toggles panels, radio, services, camera, help (§12)	

Superseded: the ☰ menu (text dropdown list) and the separate  radio icon, described in an earlier version of this section, have been fully removed and replaced by the unified icon bar from §12 — same buttons as on desktop, also at the bottom (see §17.3 for the back-and-forth on this positioning choice).





Captured under Chromium device emulation (iPhone 13, touch). Shows the full set of flight controls (joystick, roll, boost,  look) and the HUD icon bar along the bottom, plus the radio welcome message and a ship label with its target ring.

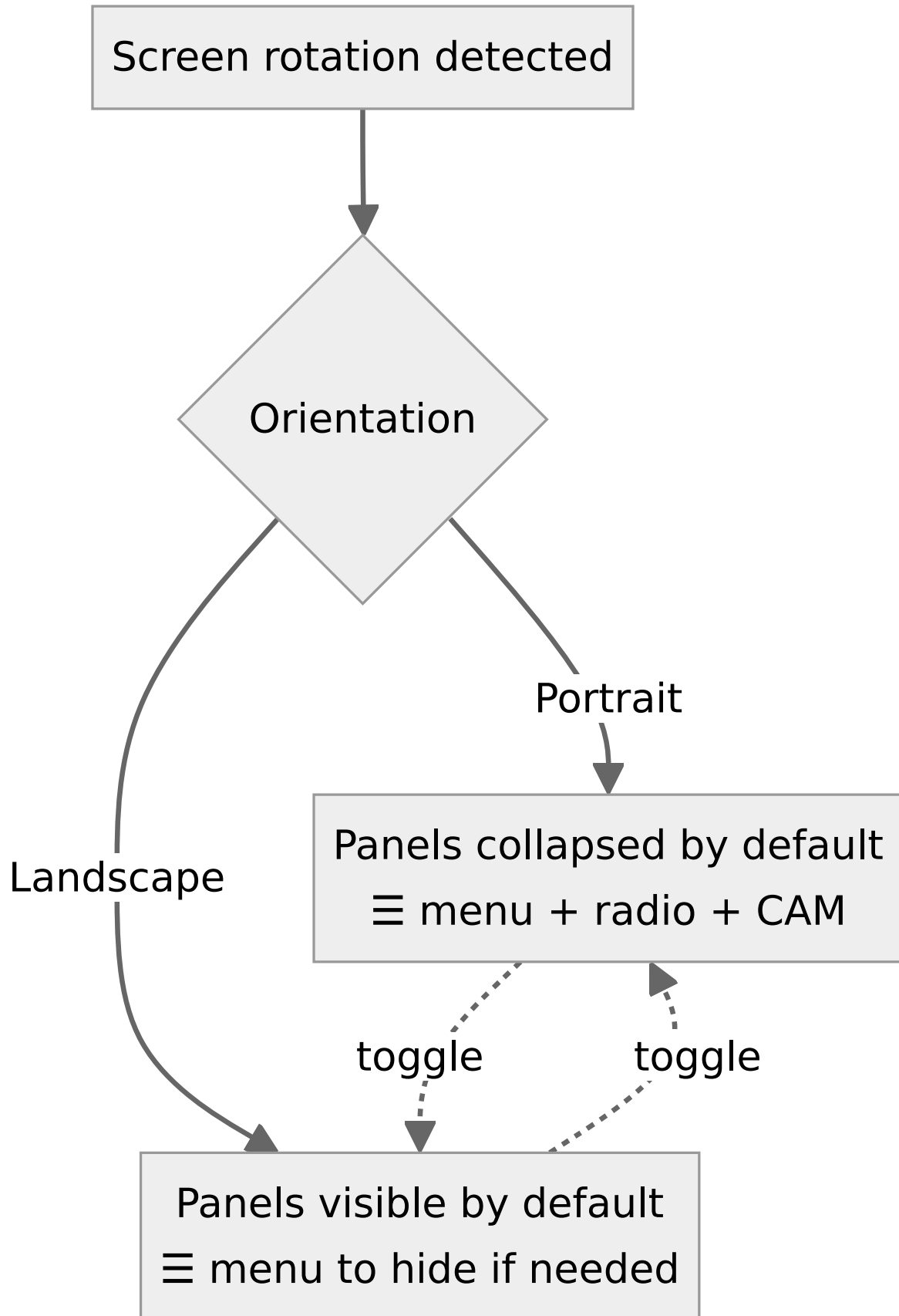
15.3 Landscape layout (tablet) – updated: unified with §12

Superseded by the HUD control bar implementation (§12): the old separate touch  menu, described in an earlier version of this section, has been fully removed and replaced by the same SVG icon bar as desktop — same icons, same activation logic. Its position changed twice along the way (top, then back to the bottom — §17.3); it now stays aligned with desktop in both orientations.



Captured under Chromium device emulation (iPad Pro 11", touch, landscape). Info panels stay visible by default in this orientation (more room than portrait); flight controls occupy the bottom of the screen, the HUD icon bar centered beneath them.

15.4 Orientation-based layout – implemented



Recomputed on `resize` and on `screen.orientation.change`, without resetting any game state — same principle as desktop window resizing.

15.5 Points still to be settled – status after refinement (§17.3)

- One-handed mode (joystick + boost grouped on the same side): not implemented.
- ~~Separate toggle for drag-to-look~~: **settled** — a dedicated button (👁), held down, now gates drag-to-look (§17.3), rather than an unconditional drag outside the control zones.
- Dedicated touch trigger to skip a maneuver (equivalent of SPACE/ENTER/ESC): **deliberately not wired** to the BOOST button, to avoid mixing thrust and maneuver-skip without an explicit decision — remains a button to add separately if wanted.

16. Port services – implemented

An area entirely absent from the previous version of this spec (just a label in the implementation-plan diagram, §18). Scope chosen: a **real, self-contained mini-feature**, rather than an empty shell waiting on the repair system (§13, still not implemented) — accumulated credits had no outlet until now.

"Thrusters" upgrade: up to 4 levels, each increasing effective cruise speed by 9% ($\text{effectiveCruiseSpeed}() = \text{CRUISE_SPEED_BASE} \times (1 + \text{level} \times 0.09)$), cost growing $\times 1.55$ per level (starting at 3,200 credits). Permanent gain, kept for the whole game — reset only by returning to the title screen (§11), consistent with a new seed.

Two new services proposed (v2.5, not implemented): this panel is now also the natural place for two further purchases, each detailed in its own section — **fuel refueling** (§22, on demand, priced by the volume missing) and **quantum jump module** (§23, one-time purchase). All three live in the same panel rather than opening a new one per feature.

Conditional activation (explicit request): the HUD bar's dedicated icon (§12) stays greyed out and inert unless one of two conditions holds, re-evaluated continuously:

- a delivery is in progress (`orbitState.active`);
- the ship is within 4 planetary radii of a port planet among those currently built (§2, `ensureSystemsBuilt` window).

If the window closes while the panel is open (the ship has moved away), it closes automatically rather than leaving a now-unreachable service on screen.

Positioning: stacked directly under the "nearest object" panel, with the radio channel (§7) taking its place below — order reversed from an overlap observed in play (see §17.1).

17. Fixes and improvements (post-Tier-1 refinement)

A batch of fixes and improvements surfaced after a first playtest pass on Tier 1. Grouped here rather than scattered across each original section, to keep a clustered record of what was observed and fixed.

17.1 Bugs fixed

Planet and star spacing. Generated systems looked too cramped. Widened spacing: minimum planet-to-planet margin (`GAP_MIN`) 260→400, additional range (`GAP_RANGE`) 360→480, orbital growth (`ORBIT_GROWTH`) $\times 1.55 \rightarrow \times 1.65$, first orbit 550-850→780-1160 units. **Cruise speed raised proportionally** (`CRUISE_SPEED_BASE` 42→56, +33%) so travel time doesn't lengthen to match — the two go together, per the history already present in the code of an earlier, opposite tuning pass (systems tightened to avoid ~16-minute trips). Re-validated via Monte-Carlo search (6 routes, random seeds): star/planet safety margins always positive, estimated cruise time 4 to 9 minutes.

Planets invisible in the flight plan. Cause: `ensureSystemsBuilt()` only built the current step and the next one — any other planet listed in the flight plan simply had no 3D mesh until reached. Window widened to `[index-1, index, index+1, index+2]`.

Port services / radio channel overlap. Two combined causes: (1) both panels were repositioned only when opened, never when one's content later grew (the radio log, notably) — each repositioning function now also triggers the other; (2) stacking order reversed (services first, then radio), per explicit request.

17.2 Polish improvements

Close button on every panel (including radio channel, port services and help) — a single delegated handler (`data-panel-cls` on each button) rather than a duplicated listener per panel.

Dynamic camera icon: three distinct symbols depending on the active mode (chase / tracking shot / distant shots), replacing a single generic pictogram.

Keyboard help as a centered overlay (`H` key): replaces the old fixed bottom bar with a centered panel, same visual family as Pause mode (amber corners, dimmed background). Content generated dynamically from the HUD button list (§12), so it always stays in sync with it.



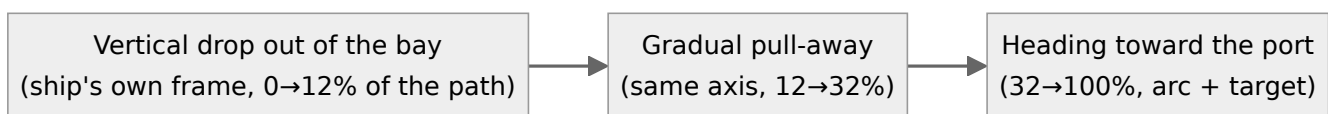
A duplicate was spotted directly on this screenshot while taking it (the H key listed twice) and fixed on the spot: help was self-listing once via the generic loop over HUD buttons, then a second time via its own dedicated description line.

"Flight plan" panel width reduced from 300 to 230px.

Radio welcome message: the captain greets the crew right after the first route is computed, before any stopover — same panel and same voice as the rest of the radio channel, no separate mechanism.

Avatars in the radio channel: captain silhouette (amber) for the crew, tower antenna (cyan) for ground control, inserted before each log line.

Shuttle launch trajectory redesigned in three phases, rather than a direct path from instant zero:



The ship's reference frame (dock position/orientation) is re-evaluated on EVERY frame during the first two phases: the cargo ship keeps maneuvering in orbit throughout the sequence, so the drop and pull-away must stay tied to its actual position, not a snapshot frozen at launch. The return to dock keeps the original simple path (not covered by the request).

Thruster trail on the shuttles: a polyline in WORLD coordinates (added to the scene, not to the shuttle's group), fade achieved by darkening colors under additive blending rather than a per-vertex alpha channel (not simply supported by `LineBasicMaterial`).

Labels + target ring on ships (cargo ship and shuttles), same visual vocabulary as stars/planets (circle + arrow + text), but with one underlying difference: the ring is sized **dynamically based on apparent on-screen size** (projecting the real radius through the camera's field of view), so it actually surrounds the object rather than being a fixed-size dot — adapted from an explicit request. Third color (green) to set them apart from stars (cyan) and planets (amber).

Object rendering in the "nearest object" panel: a 2D canvas icon (52px), not a full second 3D render — far cheaper for a thumbnail this size. Gradient tinted by planet type (dedicated palette per kind: ocean, desert, ice, volcanic, continental, gas giant with bands), true blackbody color for stars, diffuse gradient for nebulae.

17.3 Second pass – 4 additional fixes

A new batch surfaced after a second playtest session on the above deliverables.

Panel overlap, a more general cause than the one already fixed in §17.1. The left column (`hud-left` → `routePanel`) had its own **fixed** CSS offset (`top:210px`), never recomputed when `hud-left`'s actual height changed (wrapping on a small screen, panel hidden via the HUD bar, language change...) — exactly the same kind of flaw already fixed for radio/services, but left in place on this column. A single function, `repositionAllPanels()`, now handles ALL repositioning of

the panel stack (measuring each anchor's real bottom edge, never a guessed value), called at four moments: opening, toggling a panel (icon, shortcut or close button), window resize, **and** continuously (throttled to 350ms, inside the HUD update loop) — this last one to absorb height changes caused purely by content (nearest-object text, flight-plan length), which no discrete event captures.

Touch look button restored. A dedicated button (👁 icon), **held down**, now gates drag-to-look on the 3D view — a faithful equivalent of holding CTRL on keyboard, where an unmodified drag had been in effect until now (§15.1). Without the button held, a touch drag now does nothing: no piloting (already excluded, the joystick remains the sole flight control), nor look — an accidental drag can no longer trigger anything by mistake.

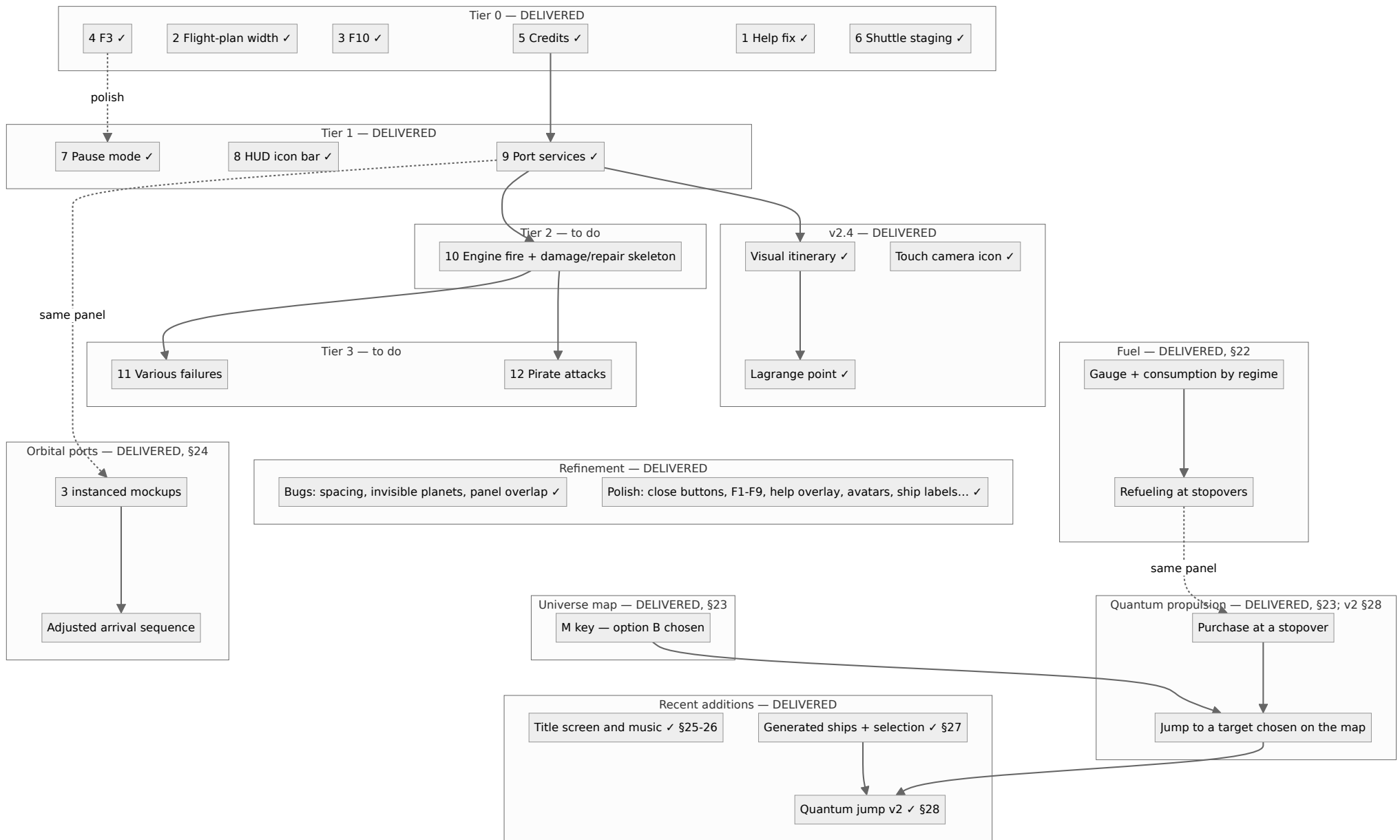
Icon bar moved back to the bottom, touch included. Reverses the choice made in §12/§15.3 (moving it to the top for lack of space) — explicit request to keep it consistent with desktop, regardless of platform. Flight controls (joystick, roll, boost, look) are shifted up by about forty pixels to make room for it right at the bottom, without overlapping.

Help button added to the bar. Tenth icon (question mark), opening the same overlay panel already in place (title + close button, §17.2) — until now reachable only via the H key, with no button to discover it. A special case in automatic shortcut assignment: keeps H rather than the F10 it would get under the usual logic (one F per button, in order), to avoid conflicting with voice mute already on F10.

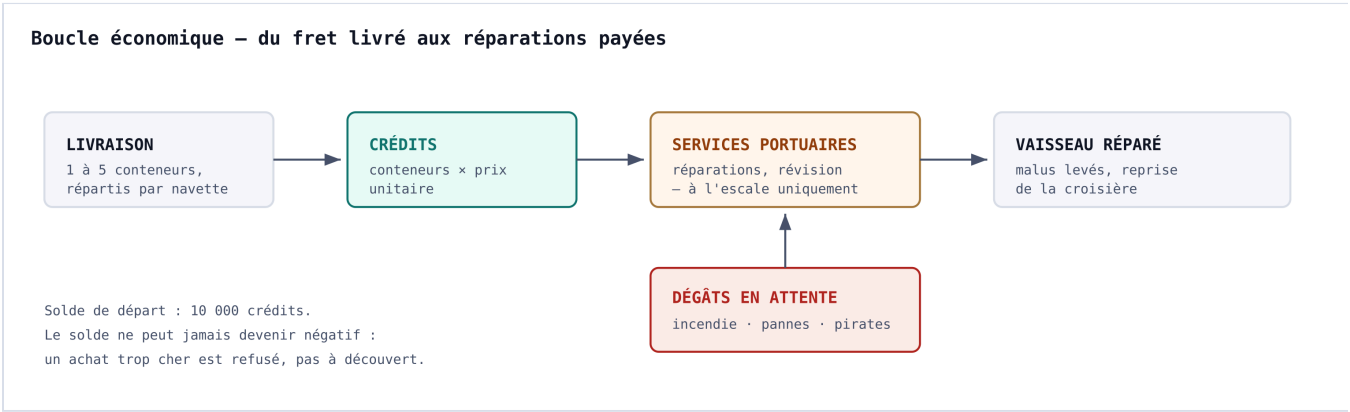
Robustness found while testing (unrelated to an explicit request, but fixed along the way): `setPointerCapture()` can throw in certain edge cases (pointer already released, multi-touch) — uncaught, it interrupted the rest of the handler before the button's state was updated. Defensive `try/catch` added on the three held touch buttons (roll, boost, look).

18. Implementation plan

Tier 0 from the previous specification (six steps with no dependency) and **Tier 1** (Pause mode, HUD bar, port services) are now **fully delivered**, with a bonus batch of fixes and polish (§17). The following tiers remain to be done.



Game-loop economy — credits now have a real outlet (§16) in addition to future repairs:



19. Universe map – implemented (M key)

*Proposed in v2.5, implemented together with quantum propulsion: it is the 3D star map described in §23.3, opened with **M** (or **;** on a QWERTY keyboard). The options evaluated at the time are kept below for reference.*

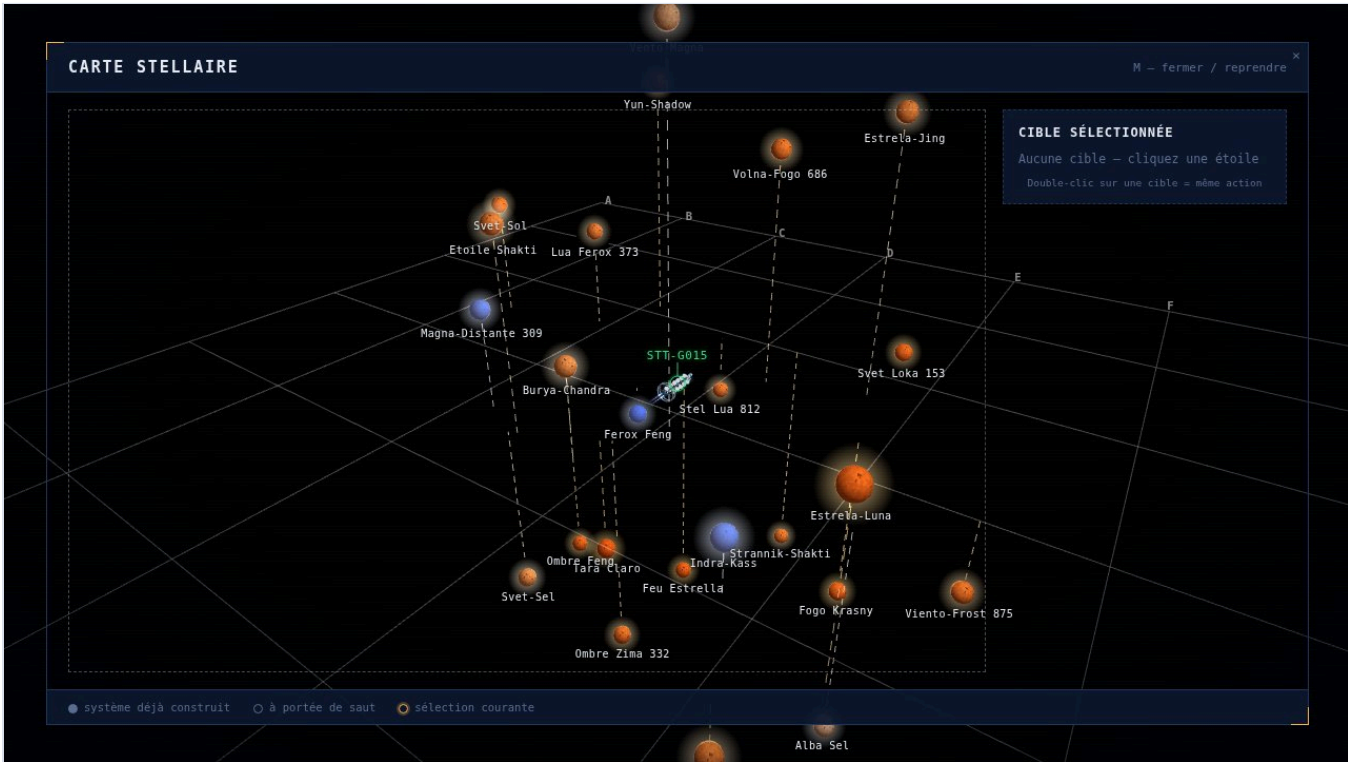
Option	Effort	Value	Detail
A — Schematic 2D map, not navigable	Low	Medium	Orthographic projection of the known flight plan (visited + upcoming steps) onto a 2D subway-map-style plane, current system highlighted. No interaction beyond show/hide. Quick to ship, but only shows what's already in the flight plan — no discovery.
B — Navigable 3D map, free camera <i>(chosen — see decision below)</i>	Medium	High	3D overview (orbital camera independent from the game's own) of every system already encountered/built, with mouse zoom/rotate. Largely reuses existing rendering (same simplified star meshes) rather than inventing a separate representation — contained cost. Gives a genuine sense of scale and exploration.
C — Full procedural galactic map	High	Uncertain	Generates and displays a region far larger than what's actually been visited (beyond built systems). Appealing on paper, but raises non-trivial questions: how far to generate without spoiling the surprise of discovery? Non-negligible generation cost at that scale for a simple overlay.

Decision (v2.5): option B, now serving double duty rather than plain viewing — the quantum-propulsion request (§23) gives it a second role, jump-destination selector. The functional detail (systems shown, click to select, pause during use) is specified in §23 rather than duplicated here, to keep the map's behavior described in one place only.

The three questions raised in the original proposal are therefore settled in §23: systems shown (built **and** within jump range, not just the flight plan), click-to-select (now essential, not just a "nice to have"), and the game pausing during use (rather than a simple toggle overlay like help) — a destination choice deserves not drifting through space while you hesitate.

Today. The map can be viewed with any ship. On a long-haul ship fitted with a jump generator, it is also used to pick a quantum jump destination; on the others, a box explains why no jump is

offered — "long-haul ships only" or "module available in port" (§28.1).



20. Reference – all keyboard shortcuts

Key	Action	Status
↑ ↓ ← → / ZQSD / WASD	Orientation	current
E / R	Roll	current
SHIFT / SPACE	Thrust	current
Mouse drag	Fine aim	current
CTRL + drag	Free look	current
CTRL (short press)	Recenters free look	current
F1 to F6	Toggles each of the HUD's 6 info panels	current
F7	Shows/hides the radio channel (replaces TAB , kept as an alias)	current
F8	Opens/closes port services (§16) — inert outside zone/delivery	current
F9	Next camera mode (replaces the old F3)	current
TAB	Shows/hides the radio channel (historical alias of F7)	current
H	Opens/closes the centered help overlay (§17.2)	current
I	Opens/closes the itinerary panel (§21)	current (v2.4)
L	Opens/closes the Lagrange-point panel (§21)	current (v2.4)

Key	Action	Status
F10	Mutes/restores the synthesized voice	current
SPACE / ENTER	Skips the approach maneuver · Resumes from pause	current
ESC / P / PAUSE	Pauses the game (§11)	current
ESC (while paused)	Returns to the title screen, full reload	current
M	Star map / quantum-jump destination selection	✅ implemented (§19, §23)
J	Intervene on an engine fire	pending (§13)

Key conflict resolved (v2.5). v2.4 had flagged a collision: **I** was already reserved (§13, never implemented) for an engine-fire intervention, while the same key had just been taken for the itinerary (§21, which is actually implemented). Resolved by moving the engine-fire reservation to **J**, which was free. **I** therefore stays assigned to the itinerary with no reservation attached.

Context-dependent keys

Key	Meaning by context
SPACE	Thrust (cruise) · Skip (approach) · Resume (pause)
ESC	Pauses · Returns to the title screen (if already paused) — no longer ever acts as a skip , that role removed to eliminate any conflict with pause
CTRL	Free look while held and dragged · Recenters on a short press alone

21. Visual itinerary and Lagrange point – ✅ implemented

Two new HUD panels, added in v2.4 with their own icons in the HUD bar (§12) and fixed-key shortcuts rather than an automatic F-slot (**I** and **L** — see §20 for the **I**/**J** arbitration).

Touch look icon. In passing, the touch free-look button (§15) changes pictogram — a camera instead of an eye — with no change in behavior: it's still the same button, held down to enable drag-to-look, the touch equivalent of **CTRL** + mouse.

21.1 Itinerary (bottom right)

A second view of the route, complementing the textual flight plan (§5): rather than a list of port names, a **visual track** with a dot representing the ship's actual position along the path.

- Sliding window of 6 steps (same logic as the flight plan, §5): spaced **evenly** along the track, not to the real scale of distances — two stars can be nearly adjacent or at opposite ends of the map, a proportional layout would make most steps unreadable.
- Three-value color code: **blue** (step completed), **amber** (current step, enlarged dot), **green** (step upcoming).
- The ship dot (white, glowing) moves continuously between two dots, interpolated along the curvilinear abscissa `ROUTE.s` already maintained by route tracking — no new background

computation, just reading it every frame (unlike the dots themselves, rebuilt only when the step changes).

21.2 Lagrange point (bottom left)

A compass giving a heading to follow independent of the main route: the ship's orientation relative to a fixed point of the current step, plus its distance.

Assumed approximation, not an N-body simulation. The targeted point sits on the star→planet axis of the current step, at 92% of the distance starting from the star — a plausible position for an L1-type point, chosen because generated planets have no simulated mass to derive a true libration point from. Shown as a secondary navigation aid, not as scientific data.

- Rotating needle (SVG dial) pointing the point's RELATIVE bearing from the ship's nose (0° = straight ahead), recomputed every frame from the ship's heading and the direction to the point — same construction as the heading already shown top left (§8), not a second coordinate system.
- Distance to the point shown in game units (u), same convention as the rest of the HUD (planetary radii, stellar distances).

21.3 Visibility

Both panels follow the unified mechanism from §12 (.panel-hidden , close button, bar icon) and start visible on desktop like the six existing info panels. **Hidden in touch mode**, regardless of orientation: their bottom-left/bottom-right slots exactly overlap the joystick, roll buttons and BOOST button (§15) — there isn't yet room to integrate them properly into that layout, unlike the info panels, which simply collapse in portrait without occupying the flight-controls' space.



22. Fuel and consumption – implemented

Implemented in `space-travel.html`. Like the other efforts of this scale (§13, §19 in their time), it first went through a detailed proposal and wireframes (§22.5) before being built — the decisions left open at that stage are settled below.

22.1 Why these numbers, not others

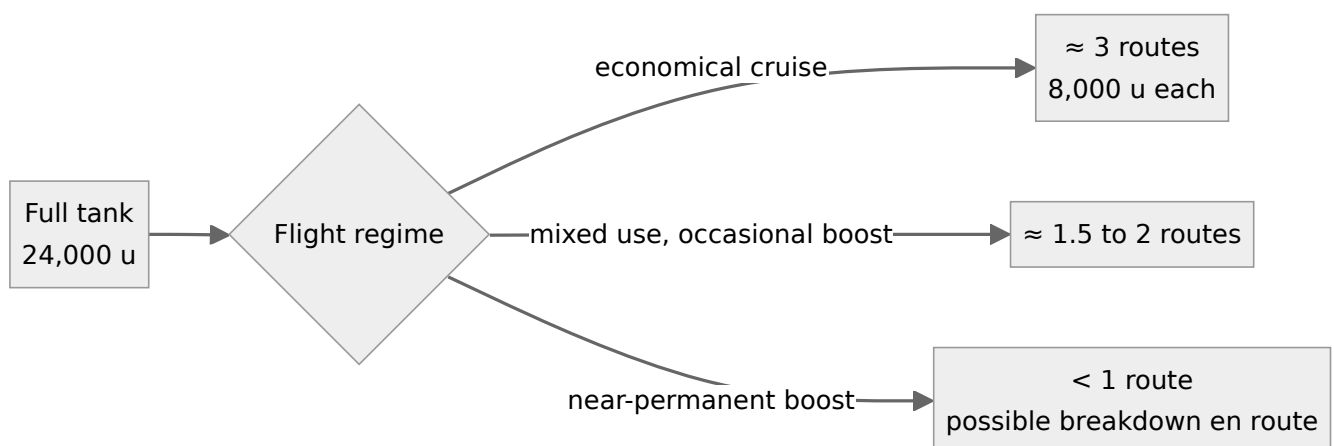
The explicitly flagged risk — stranding the player as early as the first step — isn't guarded against by picking a reserve that "feels comfortable": it's computed against the route generator's real constants, already in place and verified (§5).

- A hop between waypoint stars measures between `HOP_MIN=700` and `HOP_MAX=1500` units (`findNextWaypoint`) — **1,100 u** on average.

- A route has between 3 and 12 steps (`legCount = 3 + random(10)`, `planRoute`) — **7.5 steps** on average.
- A full route therefore averages $7.5 \times 1,100 \approx \mathbf{8,250\ u}$, rounded to **8,000 u** for the calculations that follow (intra-system legs, from the star to the target planet, stay small by comparison — a few hundred units, §6).

22.2 Gauge and consumption

- **Starting tank:** 24,000 u — the equivalent of three full routes **at economical cruise**. Resets to zero only via a return to the title screen (§11), like the propulsion levels (§16).
- **Consumption tied to engine regime**, not distance alone: `consumption(u of distance) = distance × (speed / effectiveCruiseSpeed())^1.5`. At normal cruise (the ratio is 1 by construction — a "Thrusters" upgrade, §16, therefore does NOT change a trip's fuel cost, only its duration), an average route costs the 8,000u reference figure. Under sustained boost (`BOOST_MULT=3.1`), the ratio rises to $3.1^{1.5} \approx \mathbf{5.46}$ — a route flown entirely under boost would cost $\approx 43,700\text{u}$, more than a full tank: deliberate, sustained boost is meant to stay a supplementary resource, not a cruising regime.
- **Hence the requested "1 to 3 routes"**: economical cruise end to end → up to 3 routes on the starting reserve; generous use of boost → well under one. The dial is in the player's hands, not a fixed tier.
- **First-step safeguard:** the worst isolated case — a single `HOP_MAX=1500 u` jump, flown entirely under boost — costs $\approx 1,500 \times 5.46 \approx 8,190\text{u}$, a third of the starting tank. Even an unlucky first jump (the longest possible) flown with zero restraint still leaves two-thirds of the tank intact.
- **Empty-tank behavior, settled at implementation** (not detailed in the original proposal): the ship isn't brought to a complete stop at 0u, which would leave the game unwinnable with no hope of reaching a port — boost becomes unavailable and cruise drops to 20% of normal speed (`FUEL_EMPTY_SPEED_SCALE`). Enough to limp to the nearest stopover, not enough to keep flying normally.



22.3 Refueling at stopovers

Added to the port services panel (§16), alongside the "Thrusters" upgrade rather than in a separate panel:

- Proposed rate: **0.20 credit per missing unit** — a full refill from an empty tank would cost 4,800 credits, the same order of magnitude as the first propulsion level (3,200 credits) rather than a negligible or crushing purchase.
- Charged only on the volume actually missing (just as one wouldn't pay for fuel already in the tank), not a flat fee.
- Same activation conditions as the rest of the panel (§16): stopped at a port or during an ongoing delivery.

22.4 HUD display – decision: merged with PROPULSION

An additional gauge, in the spirit of the existing PROPULSION panel (§8): bars or percentage, turning amber below 25% then red below 10%. **Decision (option B from the wireframes, §22.5):** merged into the existing PROPULSION panel rather than a standalone one — a fifth vertical gauge added after it, same visual family as the M1-M4 reactor bars (separator, track, fill), only the color changing below thresholds rather than a new panel and a new HUD icon to maintain for information read at the same moment as engine regime.

22.5 Wireframes

The two approaches considered before implementation, laid out side by side to settle the question before coding rather than after — option B (on the right) was chosen, see §22.4.

Chapitre 22 — maquettes filaires : affichage du carburant

structure et emplacement seulement — teintes, icônes et polices à trancher à l'implémentation

Option A — panneau dédié
nouveau panneau, à côté de PROPULSION

+ icône dédiée dans la barre du HUD (§12), même mécanisme de fermeture/masquage que les six panneaux d'info existants.

Option B — fusionné avec PROPULSION
une jauge de plus dans le panneau existant (§8)

+ une seule icône HUD pour les deux informations, puisqu'elles se lisent déjà au même moment (le régime moteur pilote l'une et l'autre, §22.4).

Question ouverte (§22.4) — à trancher avant implémentation

Les deux options sont fonctionnellement équivalentes ; le choix dépend surtout de la charge visuelle du HUD une fois toutes les options de propulsion et de saut quantique en place. L'option B réduit le nombre de fenêtres, l'option A garde le panneau PROPULSION inchangé et plus facile à faire évoluer séparément (§16, §23).

22.6 Overlap with the itinerary panel – bug found and fixed

The port services panel (§16), now two lines taller (fuel, in addition to thrusters), could overlap the ITINERARY panel (§21) on a short window (spotted at 1024×600, with port services and the radio channel both open at once) — the overlap check already in place (`repositionCornerBottomPanels` , §17.3) only covered the centered bottom row, not this right-hand stack, which can now reach further down. Extended to the same mechanism rather than a separate function to maintain, with

one nuance: lifting enough to clear an obstacle can, if that obstacle itself sits high on screen, push the panel RIGHT INTO it rather than out of it — hence a short loop (up to 3 passes) that re-checks the actual position after each adjustment, against both sources of overlap each time, rather than a single calculation assumed valid in one shot. On an extremely short window with many panels open at once, a safeguard keeps the panel from being pushed off the top of the screen as a last resort, at the cost of a residual overlap rather than a half-invisible panel.

23. Quantum propulsion and star map – implemented

Revised in v2.13 and v2.15: the jump is reserved for long-haul ships fitted with a jump generator (§28), hardware distinct from the warp rings of the FTL cruise (§29), and its visual effect has been entirely redone. Subsections 23.1 (purchase in port) and 23.5 (visual effect) describe the previous state, kept for reference.

Implemented in `space-travel.html`. Brings together two requests that only make sense together: a jump module is useless without a way to choose its destination, and the universe map (§19, proposed but never settled) finds its true purpose here rather than as a mere viewer.

23.1 Acquisition

One-time purchase at a stopover (port services panel, §16), not a tiered system like the thrusters (§16) — the ship either has the module or it doesn't. Price: **18,000 credits**, a structural purchase rather than an incremental upgrade, consistent with the fact that it opens an entirely new capability rather than a simple percentage gain. Once bought, the panel row permanently switches to "ACQUIRED" (same family as the other two services' "MAX LEVEL"/"FULL") — no going back short of a new game (§11).

23.2 Fuel cost of a jump

A jump consumes **exactly the fuel that the same trip would have cost at normal cruise** (§22 — $\text{distance} \times 1$, the "cruise" regime of the consumption formula), no more, no less. The module's value therefore isn't saving fuel, but saving **flight time** — a jump is near-instantaneous (§23.4) versus several minutes of simulated flight.

23.3 Star map (M key) – deliberate departures from the proposal

Keeps the spirit of option B from §19 (navigable 3D map, free orbital camera) and settles its previously open questions, with two adjustments made during implementation:

- **Dedicated scene, not the real star meshes:** the proposal envisioned reusing the meshes already in place (shaders + additive halo). In practice, a **separate Three.js scene** with simple spheres (solid for an already-built system, wireframe otherwise) proved far cheaper to build for ~24 candidates, and closer to the wireframe (§23.6) than a full stellar render would have looked at that scale. Rendered in place of the game scene while the map is open (same canvas, not a second WebGL context), through the same switch as Pause mode (`gamePaused`).
- **List capped at the 24 nearest:** jump range (§23.4) widened to a 5-cell radius would otherwise offer several hundred candidate stars — unreadable. Candidates are sorted by actual

distance and truncated, not by the cell grid alone.

- **Hand-rolled orbit camera:** spherical coordinates (drag to orbit, wheel to zoom) around the ship's position, on the same principle as free look (§4) — `THREE.OrbitControls` isn't loaded in this project.
- **Systems shown:** those already built (§2, `ensureSystemsBuilt`) **and** any star already generated within jump range (§23.4), not just the current flight plan.
- **Selection:** click a star to mark it as the target (via raycasting), explicit confirmation (button or double-click) before triggering the jump.
- **While consulting it:** the game **pauses** (reuses `gamePaused/enterPause()`, with the map's own overlay in place of the pause one) — not a simple toggle overlay like help. `M` closes the map and resumes the simulation; a dedicated key rather than overloading `ESC`, which would otherwise have inherited its "quits to title screen if already paused" behavior.
- **Without the module:** the map stays browsable read-only — the confirm button reads "MODULE REQUIRED" rather than hiding the feature. Insufficient fuel for the targeted star gets the same treatment, "NOT ENOUGH FUEL".
- **Dedicated HUD icon** (alongside the `M` key), added during implementation: without it, the map would have stayed completely out of reach on touch, with no keyboard available.

23.4 Jump range and regeneration

A jump can only target a star **already generated** by the cell-based star field (§2) — computed directly via `starDataForCell()` (a pure, deterministic calculation) rather than depending on the stars already rendered (a much narrower window, §2). After a jump:

- The star field and nebula refresh around the new position (reuses `refreshField`, already in place for normal scrolling).
- The current route is **replanned from the new position** (reuses `computeRoute`, without the nose-to-curve realignment reserved for the very first departure).
- The arrival point isn't the targeted star's center but a safe point along the targeted axis, at a distance proportional to the star's actual radius (never less than 400u).

23.5 Visual effect — implemented by reusing what's already there, not by stretching geometry

The underlying goal stands (an effect that clearly reads as a jump, with no costly new rendering system), but the implementation notably simplifies the middle phase compared to the original idea:

1. **Charge-up (≈0.4s):** the four engines (`M1-M4`) exceed their normal light intensity, a pulsing cyan halo (new `.jump-vignette` element) grows around the screen.
2. **Jump (≈0.5s):** rather than individually stretching each star in the field along the camera→star axis (costly to implement cleanly), the effect directly reuses `.speedlines` — the radial halo already in place for boost (§4) — pushed to a much higher opacity. The visual result (radial streaks, warp-speed style) stays faithful to the intent, for a fraction of the implementation cost. The actual teleport (position, fuel, field regeneration, route replanning) happens midway through this phase, masked by the effect's intensity.

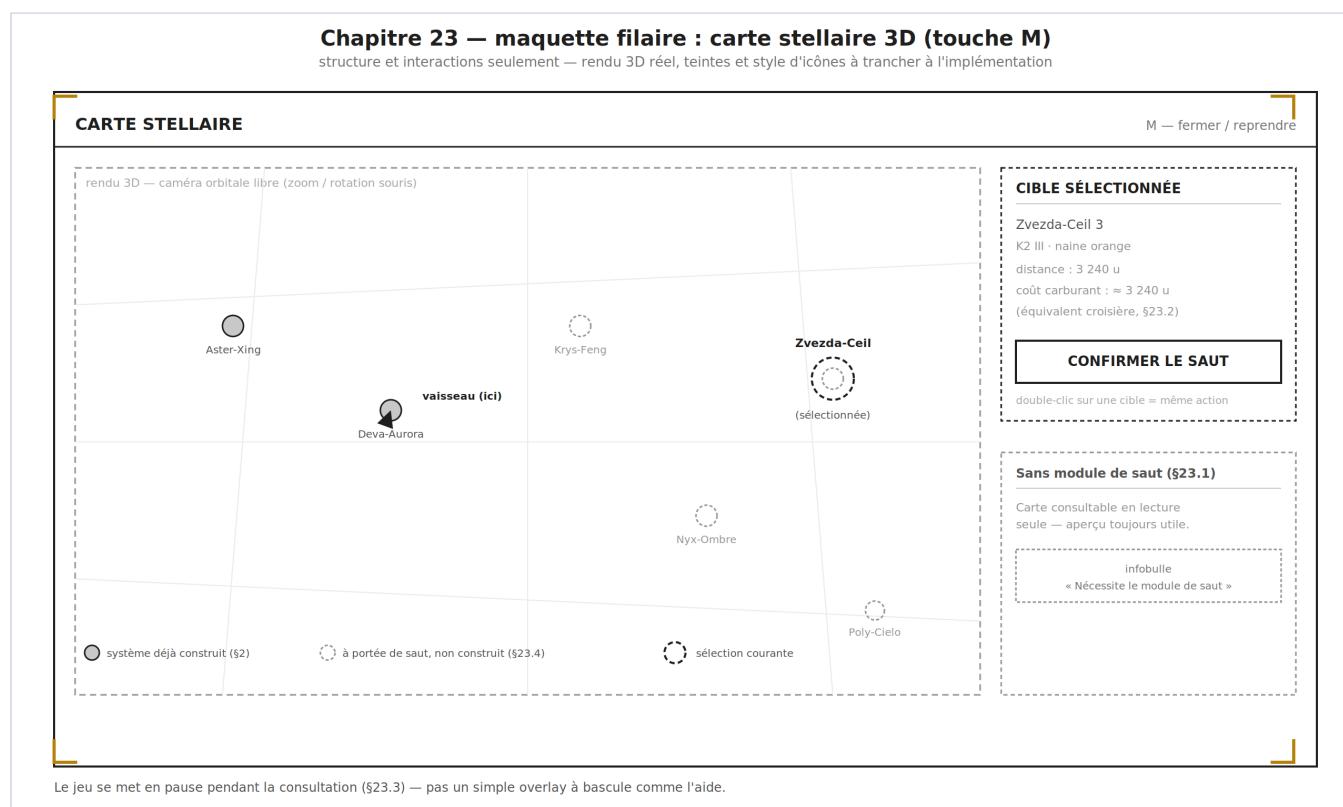
3. **Exit (≈0.15s)**: a brief white flash (new `.jump-flash` element, opacity 0→1→0) marks arrival. Camera fixed throughout the sequence: normal flight control (`updateFlight`) is entirely bypassed while the jump is in progress, non-interruptible like orbit insertion.
4. HUD banner "QUANTUM JUMP IN PROGRESS" during the sequence, on its own element (`#jumpBanner`) rather than the existing mode badge, which normal flight logic keeps rewriting.

23.6 Two bugs found and fixed during implementation

- **Malformed HTML comment**: a code comment ended up closed with `*/` (JS/CSS syntax) instead of `-->` — the browser then treated everything up to the next real `-->` as comment content, quietly making the map panel vanish from the DOM (present in the source, unfindable via `getElementById`). Spotted by comparing two seemingly contradictory symptoms (the string present in `innerHTML`, the element missing for `getElementById`) rather than assuming a script-side problem.
- **Overlay nested in the wrong place**: the map and jump banner were first placed inside the general `#hud` container. Once a rule was added to hide that container while the map is open (to clear the screen of old panels, frozen since their refresh loop stops with the rest of the game), that same rule also hid the map itself, a descendant of it. Fixed by moving these elements out of `#hud`, as `#pauseOverlay` already does.

23.7 Map wireframe (historical reference)

Wireframe produced before implementation, to settle the general structure — the actual rendering (§23.3) simplifies some choices (spheres rather than full meshes), for the reasons explained above.



24. Orbital ports

Proposal, not implemented. A second port type, complementing — not replacing — the existing ground port (§6, §7): some ports float in orbit around a planet rather than sitting on it.

24.1 Why a second type

Beyond visual variety, a concrete case the current system doesn't handle properly: a **gas giant** (one of the six planet kinds, §6) by definition has no solid surface to place a port on. If a gas giant is ever chosen as a step's target planet, the orbital port isn't one option among others but the **only** coherent one — an implicit blind spot in the current system that this effort closes along the way.

24.2 Generation: a small, instanced set of mockups

Rather than a 3D model generated on the fly per port (costly, and little reason to vary a station endlessly the way a planetary system is varied): a **small catalogue of fixed mockups** — starting proposal, three models —, chosen and positioned at system generation like everything else (§2), then **instanced** (same base mesh reused, only the transform changes) everywhere they appear, in the same spirit already used for asteroids or star fields.

Model	Silhouette	Ports concerned
Simple ring	An open ring, single dock at its center	Small stopovers, secondary systems
Multi-arm hub	Central core, 3-4 radiating docking arms	Main ports, busy systems
Vertical docking tower	Elongated structure along the orbital axis	Gas giants (§24.1) — profile designed for a fast low orbit

- **Which systems get one:** seed-weighted draw (deterministic, like the rest of generation, §2) — starting proposal, about a third of systems, and **mandatory** for any gas giant chosen as a target (§24.1), which has no alternative.
- **Scale:** each mockup sized proportionally to the planet it orbits (planet radius, already available at system-generation time), not a fixed size that would look tiny in some cases and overwhelming in others.

24.3 Arrival sequence: an adjustment, not a redesign

The existing orbit system (§7 — orbit insertion, orbit ring around the target) already fits the orbital-port concept well: the station simply occupies a fixed point on that ring, rather than the planet's center. Concrete changes:

- The port itself replaces the ground delivery point as the orbit-insertion's visual target.
- Shuttles (§7) head straight for the station — a shorter, simpler trip than a surface approach, with no descent phase to stage.
- Radio dialogue (§7, §14) addresses the station rather than "the ground" — a text-template adjustment, not a new dialogue mechanic.

No screenshot for this section: like random events (§13), an effort still at the proposal stage — an illustration will come once the three mockups are actually built, not before.

25. Title screen: keyboard navigation and background flythrough – implemented

25.1 Keyboard navigation

Alongside the click/touch already in place (the language buttons are real `<button>` elements, touch-friendly by nature): left/right arrows (up/down also accepted, for tolerance) move focus between the four languages, ENTER or SPACE confirms the focused choice — never a separate path from the click, one shared function (`selectLanguageAndStart`) handles both. Dedicated focus outline (amber border), distinct from hover (cyan border).

25.2 Background flythrough

A succession of very slow shots through the already-generated universe — a star, planet, or nebula chosen at random every 14 to 24 seconds, the camera drifting toward it over several seconds rather than cutting to it instantly. HUD and targeting labels hidden throughout the title screen (`body.title-active`), leaving only space on screen — **the ship and its route trace (`ROUTE.gates`) too** (`.visible = false` on both, restored once the game actually starts): route computation is moved up to startup (next paragraph), but what that computation produces has no place in a shot meant to show the generated universe, not yet the cargo ship or its itinerary.

Practical requirement: the world has to exist before there's anything to fly through. Route computation (`computeRoute` , §5) and the first fill of the star/nebula fields (`refreshField` , §2) — until now deferred to the actual start of the game — are therefore moved up to the moment the intro hands off to the title screen. No side effect: this computation depends only on the ship's starting position/heading, never on the language choice, and `ensureSystemsBuilt()` (called at the end of `computeRoute`) builds the first planetary system along the way — without this move, no planet would exist yet for the flythrough at that point.

Never a BACKWARD dolly shot (explicit request). Each new shot originally chose a fully random approach direction around its target, unrelated to the camera's current position — which could very well place it behind its current direction of travel, forcing the next shot to back up to reach it. Fixed by ALWAYS approaching the new target from the camera's current position, along the straight line between the two, stopping at framing distance before reaching it: the camera therefore only ever moves forward, never backward. Verified across eight consecutive shot changes: perfect alignment between the motion and the direction to the new target every time.

25.3 Three bugs found while implementing

- **Stars invisible at framing distance:** in this game, a star is modeled at a tiny physical scale (a few tenths of a unit) — its apparent size comes only from an angle calculation (`baseCore×26/distance`) run every frame during normal flight, never in the new "title screen"

branch. Fixed by reusing that calculation (distance to the cinematic CAMERA, not to the ship, since the ship isn't flying yet).

- **Star framing distance miscalibrated:** even once that calculation was reused, the first attempt (a distance proportional to the halo, several hundred units) still left the star sub-pixel — the computed angle was capping at its floor. The angle calculation is only meant for a CLOSE flythrough (a few units to a few dozen, never more): fixed by basing the framing distance on `baseCore×8.7` (targeting an angle ≈ 3) instead of the halo size.
- **Title screen background stayed black despite correct rendering:** once the two points above were fixed, the screen still stayed black. Cause, found by reading the WebGL framebuffer's pixels directly (28% bright pixels even though the screenshot looked uniformly black): `#titleScreen` had an opaque, full-screen background (`background:var(--bg)`), entirely hiding the 3D canvas underneath — the exact same flaw already found and fixed on the star map (§23.6). Fixed by making that background transparent, with a soft veil and drop shadows behind the text to stay legible over a now-visible, changing backdrop.

25.4 v2.12 fixes and additions

- **Background stars too fast, passing in front of objects.** The decorative star background (several layers of distant points) was only re-centered on the *ship*'s position, which doesn't move during the title screen, while the flythrough camera travels hundreds of units: left in place, that background was flown through instead of staying at an apparent infinite distance. Fixed by re-centering it on the *camera* during the flythrough, reinforced by a very low render order that guarantees these layers stay behind everything else.
- **Legal notice.** A "(c) 2026 - game by Frédéric Delorme - music by ScoreStudio from Envato" line at the bottom of the title screen, white on a transparent background, discreet and non-clickable.
- **Music as soon as the title screen appears**, no longer only on the click that dismisses the intro (see §26.1) — including when the intro ends on its own. A limit inherent to browsers: without a prior user gesture, playback is refused; that refusal is absorbed silently and music starts on the next gesture.



26. Background music and volume controls – implemented

26.1 Music

Playback of a royalty-free track, looped, via `Audio()` rather than an HTML `<audio>` element (nothing to display, just something to drive from script). Shipped **separately** — not embedded as base64 in the main file like the rest of this project's generated assets: the music file is a plain external asset, no reason to bloat the game file with it. Loaded from a `musics/` folder next to the HTML file.

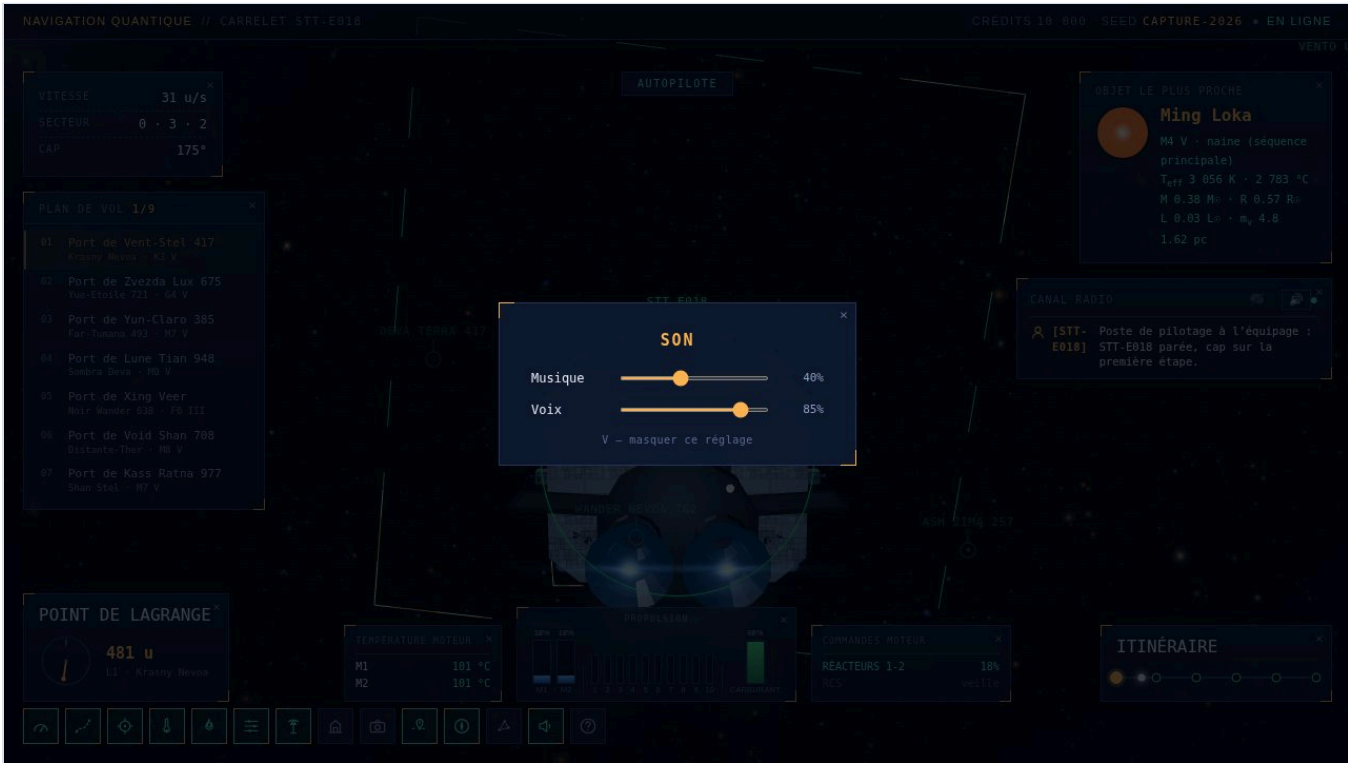
Technical point — browser autoplay policy. A browser refuses to start sound before an explicit user gesture. Playback is therefore triggered by the click that dismisses the intro (§11) — the first genuine gesture of the whole session — rather than waiting for some later moment that would, either way, still need its own dedicated gesture. Any browser refusal (`play()` rejected) is absorbed silently: the music is an amenity, never something whose absence should surface as an error.

Revised in v2.12: playback starts as soon as the title screen appears (§25.4).

26.2 Volume controls

A new small dialog (`v` key, or a dedicated icon in the HUD bar, §12) with two independent sliders — music and synthesized voice (§14) — rather than a simple mute/unmute switch like the existing `F10` shortcut (kept as-is, it still acts on the same state). The voice's volume, until now a fixed value (`0.85`) hardcoded into the construction of every utterance, becomes a setting that persists for the whole session — a single source of truth, read at each new utterance rather than fixed at dialog-creation time.

A simple toggle overlay (like help, §17.2), not a consultation that pauses the game like the star map (§23): adjusting a volume doesn't warrant freezing the flight in progress.



27. Generated ships and selection screen – implemented

Up to v2.12, the game had a single ship, drawn once and for all. v2.13 integrates the generator developed in the separate study "Generative construction of cargo ships": the player chooses a ship at launch, and that choice shows — silhouette, size, propulsion, jump capability.

27.1 Ten models

Model	Type	Tier	Length	Epstein nozzles	Options (long-haul)
Mistral	Light tug	I	61 m	2	—
Vagabonde	Fast light freighter	II	71 m	3	—
Sirocco	Medium tug	II	89 m	3	—
Carrelet	Warehouse freighter	II	96 m	2	—
Hirondelle	Spine freighter	II	110 m	2	—
Tramontane	Heavy tug	III	120 m	4	—
Basalte	Warehouse bulk carrier	IV	120 m	6	FTL cruise ×8, jump
Belle-Étoile	Liner	III	140 m	5	FTL cruise ×10, jump
Banquise	Ice tanker	IV	181 m	4	FTL cruise ×5, jump
Longue-Échine	Long spine freighter	III	237 m	4	FTL cruise ×10, jump

Lengths measured on the models. All models share the same building blocks — bow, modular habitat, engine section — built as a chain; shapes, sizing rules and the Epstein drive shader are described in the study.

27.2 Selection screen (temporary dialog)

It sits between the language choice and the start of the game; the title-screen flythrough keeps running in the background, and ship and HUD stay hidden until the choice. Temporary by design: it will be replaced by the shipyard store, which a note at the top of the dialog points out.

- **On the left**, a carousel: slowly rotating 3D preview, previous / next arrows, and a thumbnail strip sorted **from smallest to largest** — length measured on each model at first display, not a hard-coded list.
- **On the right**, the sheet: name, type, tier, registration; length, width, height; dry and loaded mass; containers or tanks; passengers and crew; propulsion; quantum jump (available or not); defense; radiators.
- **Propulsion system**: "Epstein +" two combinable toggles, **FTL cruise xN** (warp rings, \$29) and **Quantum jump** (jump generator, \$28), both on by default for long-haul ships; 3D preview, dimensions, masses and the "FTL cruise" and "Quantum jump" rows follow the choice. On other ships, both options are greyed out, with the note "FTL cruise and quantum jump reserved for long-haul ships".
- **Keyboard**: ← → to change ship, ↑ ↓ to cycle through the four propulsion combinations, Enter to board. Labels in the game's four languages.
- **Preview**: dedicated renderer with the same color settings as the game (sRGB, ACES tone mapping), released when the dialog closes.



27.3 Wiring into existing systems

The generator lives in its own namespace (SHIPGEN) and shares nothing by accident with the rest of the game. A single function, `installShip()` , connects the chosen model to everything that depended on the old ship:

System	Adaptation
In-game size	length brought to $40 \times (L/89)^{0.55}$ units, from ~ 35 u (Mistral) to ~ 70 u (Longue-Échine) — the old ship was 40: sizes read as different without the camera entering the largest ones
Chase camera, avoidance, labels	camera offset and radii proportional to that size
Engine gauges and temperatures	panel rebuilt for the actual nozzle count (1 to 6); temperatures, previously sized for 4 engines, follow
RCS thrusters	one jet per RCS block of the model, its axis (yaw, pitch, roll) derived from its position; the existing animation drives them unchanged
Lights and beacons	shared materials, existing blinking unchanged; antenna lights join the beacons
Shuttle dock and arm	original code made configurable, placed under the habitat module (under the rear bulkhead for the warehouse freighter), at game scale rather than hull scale
Epstein drive	intensity driven by actual flight thrust, boost included; glow scaled to the ship
Propulsion options	warp rings $\rightarrow$ FTL cruise (\$29); jump generator $\rightarrow$ quantum jump (\$28). Fitted independently; the generator can be added in port without rebuilding the dock
Colors	materials and textures converted for the game's sRGB + ACES rendering, without which hulls would look washed out
HUD	name and registration of the chosen ship



27.4 Masses and capacities

Orders of magnitude, to be calibrated when the store sets prices: dry mass $\approx 0.35 \times L \times \sqrt{(w \times h)}$ tonnes, +6% per warp ring, +4% for the jump generator; FTL factor = $12 - 1.4 \times \log_2(\text{loaded mass without options} / 1,000 \text{ t})$, clamped between 5 and 10 (§29.1); loaded container: 24 t; ice tank: $\approx 2,000 \text{ t}$; two passengers per cabin; crew by tier (2, 4, 6 or 10).

27.5 Points noted while implementing

- **Initialization order.** Several ship-dependent settings were constants declared further down the script: installing the default ship was moved to the end of the script, once everything is declared.
- **Swallowed Enter key.** When the dialog opened, keyboard focus stayed on the title screen's language button, and Enter was ignored. The dialog now takes focus and consumes the keys it handles — otherwise Enter could also relaunch the title screen during its fade.
- **Verified:** installation of every ship type, keyboard selection, no errors, in both the readable and the obfuscated build.

28. Quantum jump v2: spacetime distortion – implemented

28.1 Jump generator, reserved for long-haul ships

The jump relies on a **jump generator** — a glowing core inside an icosahedral cage, on a dorsal pylon at the middle of the reactor —, hardware distinct from the warp rings of the FTL cruise (§29). Only long-haul ships (Basalte, Longue-Échine, Banquise, Belle-Étoile) can carry it: chosen on the selection screen (§27.2), or bought later in port (18,000 credits, §23.1) — only the hull is then rebuilt, without touching the dock, the loading arm or a shuttle mid-operation. For other ships, the star map shows "Jump impossible: long-haul ships with a jump generator only", the port row "Long-haul ships only".

The jump **teleports** the ship to another region of the map, chosen on the star map (§19, §23) — whereas the FTL cruise speeds up travel along the current route.

28.2 Sequence (4.4 s, non-interruptible)

Phase	Duration	What you see
Charge	1.8 s	engines cut, the generator powers up (core and halo growing brighter), first ripples in space
Fold	1.0 s	space curves around the ship as under a gravitational lens — iridescence, bright ring —, the ship stretches along its axis
Jump	0.3 s	flash; teleport at the middle of the phase
Wave	1.3 s	on arrival, space relaxes, a shock wave crosses the image, engines resume





28.3 Technique

Only for the duration of the jump, the scene is rendered into a texture, then deflected full-screen by a shader: 1/r lens, slight swirl, iridescence, bright ring, shock wave. The ship's area is excluded from the deflection — it is space that folds around it, not the model. Outside a jump, the game renders once, as before: no extra cost. The v2.6 banner, veil and flash are kept.

28.4 Risks and limits

- The intermediate render has no anti-aliasing: slightly harder edges for 4.4 s, deemed acceptable.
- Actual smoothness remains to be confirmed on a real machine: the test environment, with software rendering, is too slow to judge it. The full timeline, however, was verified step by step (teleport, return to normal state).
- Cost: one extra full-screen pass, bounded to the jump's duration.

29. FTL cruise – implemented

Second long-haul option, distinct from the jump: no teleport, but a **cruise speed multiplied by 5 to 10** along the current route.

	FTL cruise	Quantum jump (\$28)
Hardware	two warp rings and their bubble	jump generator
Effect	cruise speed ×5 to ×10	teleport to another region of the map
Trigger	automatic, away from stops	destination picked on the star map
How to get it	selection screen	selection screen, or purchase in port



29.1 Factor per ship

The heavier a loaded ship, the slower its bubble: factor = $12 - 1.4 \times \log_2(\text{loaded mass without options} / 1,000 \text{ t})$, clamped between 5 and 10. Computed without options, it does not depend on the chosen equipment.

Ship	Loaded mass (options included)	Factor
Belle-Étoile	2,398 t	×10
Longue-Échine	3,752 t	×10
Basalte	6,455 t	×8
Banquise	34,231 t	×5

29.2 Engaging and cutting off

- **Engaged** progressively ($\approx 2 \text{ s}$) as long as the next stop is far enough; **cut off** when the remaining distance drops below $950 \text{ u} + \text{speed} \times 1.2 \text{ s}$. The ship is back to normal speed before the approach zone: arrival, orbit insertion and shuttles are unchanged.
- **Boost** has no effect during FTL cruise.
- **Fuel**: the bubble is powered by the reactor — consumption per kilometer stays that of normal cruise (§22); only travel time goes down.
- **On screen**: warp bubble around the ship, ring coils spinning, speed lines, factor shown next to the speed (for example "560 u/s ×10").

29.3 Issue found by the tests

The first setting cut the FTL cruise off at $950 \text{ u} + 2.5 \text{ s}$ of speed, i.e. $\approx 2,350 \text{ u}$ at ×10: on a short leg (2,390 u with the test seed), it never engaged. The automated test (§30.2) revealed it; the

threshold was brought down to 1.2 s, the actual deceleration time.

30. Project organization, build and tests – implemented

Since v2.15, the game is no longer maintained as a single file: sources are split by kind, and the single page is produced by a build.

build.js	build, tests, screenshots, PDF, packaging (Node.js)
package.json	development dependencies, npm shortcuts
src/	
html/index.html	page skeleton, markers <code>/*@inline:css*/</code> et <code>/*@inline:js*/</code>
css/style.css	styles
js/NN-*.js	game code, one file per numbered section (21 files)
assets/musics/	background music
test/	9 automated tests, harness (lib/), screenshot script
docs/	FR/EN specification, study, demo pages
target/	generated: compact page, readable build, PDFs, archive

The JavaScript code remains a classic script: its files share the same global scope and are concatenated in the order of their numbers. During the migration, reassembling `src/` was verified to be byte-for-byte identical to the original file.

30.1 Commands

Command	Effect
<code>npm run build</code>	assembles <code>src/</code> : <code>target/space-travel.min.html</code> (compact, obfuscated, three.js and music embedded, 6.8 MB), its <code>.gz</code> version, and <code>target/dev/space-travel.html</code> (readable)
<code>npm test</code>	9 tests on the compact build (<code>npm run test:dev</code> : readable build)
<code>npm run captures</code>	regenerates the screenshots of this specification
<code>npm run docs</code>	PDFs of the FR / EN specification and of the study, Mermaid diagrams rendered
<code>npm run package</code>	delivery archive <code>target/space-travel-<version>.zip</code>
<code>npm run all</code>	clean, build, tests, PDFs, archive — the archive is only produced if the tests pass

30.2 Automated tests

Test	Checks
Startup	loads without errors, title screen, legal notice, seed set from the URL
Selection	ten ships from smallest to largest, propulsion options per ship, keyboard
Ships	installation of every model: gauges, RCS, dock, arm, beacons, camera, propulsion
FTL cruise	cruise ×5 to ×10 away from stops, cut-off before the approach, consumption per km
Quantum jump	full sequence, teleport, return to normal state

Test	Checks
Port	buying the generator without touching the dock; refused outside long-haul ships
Translations	labels present in all four languages
Mobile	selection and boarding by touch, touch controls
Rendering	detailed planets and asteroids, distance-based detail, shared resources preserved, jump grid, FTL streaks

Any JavaScript error in the page fails the current test. Tests run with software rendering (SwiftShader) to work everywhere, including without a GPU: they take several minutes.

30.3 Screenshots, documents, reproducible universe

- **Screenshots:** `npm run captures` replays each situation illustrated here — cruise, panels, pause, help, approach, orbit and shuttles, star map, jump, FTL cruise, mobile, tablet — with a fixed seed, and rewrites the images under the same names.
- **PDFs:** Mermaid diagrams are rendered as graphics; a diagram shrunk too much on A4 is redrawn in the other orientation, or gets a landscape page.
- **Seed:** `space-travel.html?seed=MY-SEED` sets the universe — same stars, same routes. Tests and screenshots rely on it; it is also a way to share a game.
- **Limit:** fonts are loaded online; offline, the page falls back to system fonts.

31. Celestial rendering and propulsion effects –

implemented

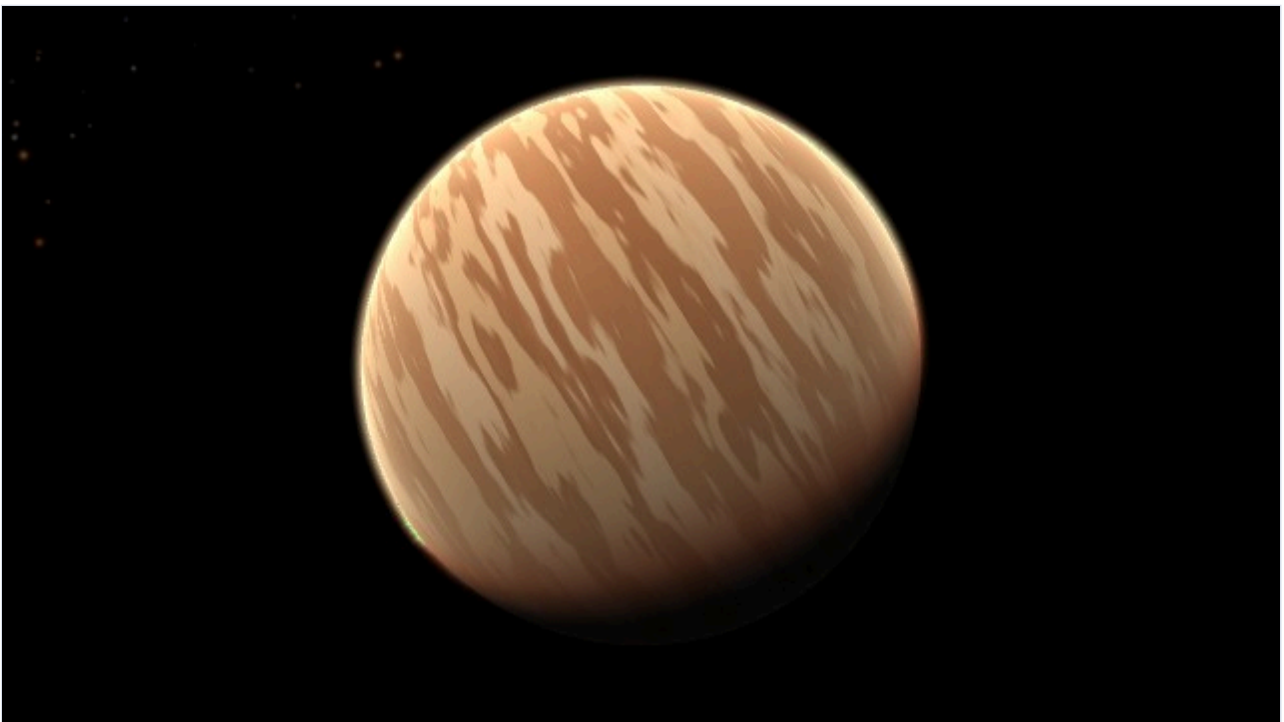
Taken from the "Observation des étoiles" demo (v4). The demo drove the v2.15 engine without modifying it — the game it embeds is byte-for-byte identical to the 2.15 compact build — and added two self-contained rendering modules and staging effects. The modules are taken as is (`src/js/14b-planetes-asteroides-detaillies.js`), the effects adapted to the game's timeline (`src/js/14c-effets-saut-supraluminique.js`).

Improvement	Hook in the game
Detailed planets	system construction (<code>ensureSystemsBuilt</code>), per-frame update (<code>updatePlanetSystems</code>)
Textured asteroids and moons	system construction; shared resources detached before destruction (<code>disposePlanetGroup</code>)
System star	star loop: disk and halo kept visible from afar for the stars of built systems
Dynamic resolution	start of each frame (<code>animate</code>)
Jump grid, FTL streaks	jump sequence, FTL cruise

31.1 Detailed planets

Procedural surface with adaptive detail (the grain follows the size of an on-screen pixel), three cloud layers each drifting at its own speed and casting its shadow, scattering atmosphere — blue

rim, sunsets at the terminator — replacing the former shell, polar auroras on part of the worlds with an atmosphere (more frequent on inhabited and icy worlds). City lights on the night side of inhabited worlds.



31.2 Textured asteroids and moons

Five rock templates (deformed icospheres: irregular shape, craters, ridges) and four material families generated at startup — carbonaceous, silicate, metallic, icy —, with seamless mapping and screen-space relief. Each belt mixes two families; moons become cratered rocks. Geometries and materials are shared between all systems: they are detached before a system is destroyed, so as not to be destroyed with it.

31.3 Distance-based detail (game-specific addition)

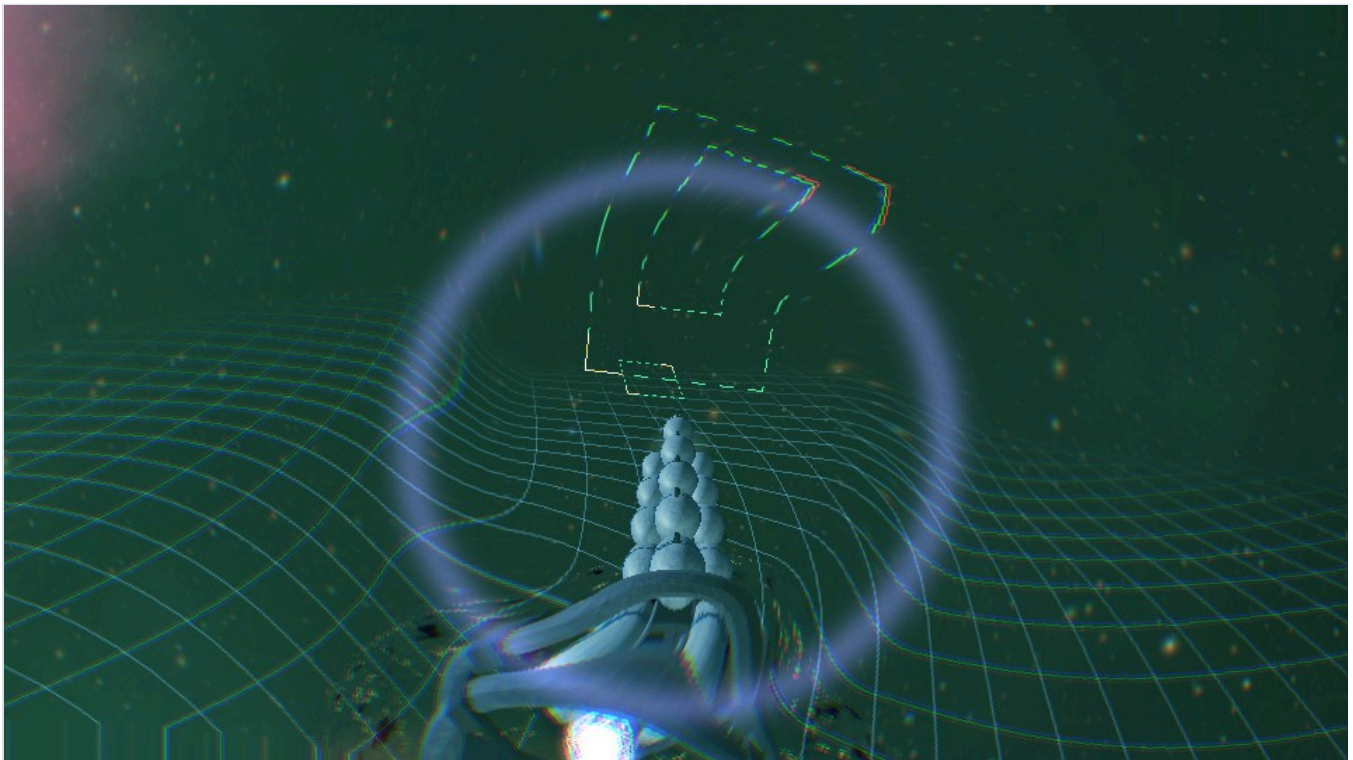
The demo films one system at a time; the game keeps up to four built. At full detail everywhere, planets would amount to several million triangles.

Apparent planet radius	Surface	Clouds	Auroras	Atmosphere
over 110 px (back down under 80 px)	192 × 144 (demo detail)	176 × 132	visible beyond 40 px	visible
up to 110 px	64 × 48	48 × 36, hidden under 6 px	hidden	hidden under 2 px

Full detail is only built on first approach, at most one planet per frame, to avoid any hitch.

31.4 Propulsion effects

- **Quantum jump:** during charge and fold, a spacetime grid sinks and twists under the ship, which slides into the well before the flash; on arrival, a new well closes back while rippling. Matched to the game's timeline (§28.2).
- **FTL cruise:** 3D star streaks scroll around the ship, their length following the speed; they replace the screen-overlay lines, kept for boost. The lens pass the demo also adds during cruise is not taken: it would cost a full-screen pass for the whole cruise.





31.5 Dynamic resolution

When frames average over 23 ms (planetary close-up), resolution drops in 10% steps, down to 45%; it climbs back slowly when frame time returns below 18.5 ms. The ceiling stays the game's (screen pixel density, up to 2). URL parameters: `?quality=low` lowers the ceiling, `?quality=fixed` freezes resolution (reproducible screenshots).

31.6 Checks and limits

- "Rendering" automated test (§30.2): improved planets and rocks in every built system, full detail up close and light sphere far away, shared resources intact after a system is destroyed, jump grid, FTL streaks. Jump and FTL cruise tests rerun without regression.
- Procedural shaders are costly without a GPU: in the test environment (software rendering), a planetary close-up at 1280 × 720 takes over two minutes per frame. The illustrations of this section are therefore rendered smaller, and the screenshots of §6-7 (approach, orbit) still show the former planet rendering: `npm run captures -- --only=img-007` regenerates them on a machine with a GPU.